

AADS Mid Sem Sandali

1. Red Black Tree
 - a. Insertion
 - b. Deletion
 - c. Search
 - d. Pseudocode for each
 - e. Time and Space Complexity
2. AVL Tree
 - a. Insertion
 - b. Deletion
 - c. Search
 - d. Types of Rotations (different cases, explain with example)
 - e. Pseudocode for each
 - f. Time and Space Complexity
3. Binary MaxHeap/MinHeap
 - a. Insertion
 - b. Heapify
 - c. Heap sort
 - d. Deletion
 - e. Find kth max, kth min element
 - f. Pseudocode for each
 - g. Time and Space Complexity
4. Binary Search Tree
 - a. Insertion
 - b. Deletion
 - c. Check if tree is binary tree
 - d. Find an element
 - e. Pseudocode for each
 - f. Time and Space Complexity
5. Self adjusting - splay tree
6. Fibonacci Heap
 - a. Insertion
 - b. Deletion
 - c. Find an element
 - d. Pseudocode for each
 - e. Time and Space Complexity

7. Binomial Heap

- a. Insertion
- b. Deletion
- c. Search
- d. Union Operation on Binomial Heap (in PYQ)
- e. Pseudocode for each
- f. Time and Space Complexity

8. Sorting (Code, PseudoCode, Time Complexity - Worst/Best, which one best, better, asc, desc order)

- a. QuickSort
- b. Randomized Quick Sort
- c. Radix Sort
- d. Merge Sort
- e. Heap Sort
- f. Bucket Sort
- g. Count Sort
- h. Bubble Sort
- i. Insertion
- j. Selection

9. Search

- a. Binary Search
- b. Linear Search

10. Sparse Matrices

- a. Properties
- b. Applications
- c. Representation All - Linked List (one more asked in PYQ)
- d. Time Complexity for all representations

11. Divide and Conquer

- a. Algorithm
- b. Applications
- c. Master's Theorem (solve PYQ only for numerical)

12. Hashing

- a. Universal Hashing
- b. Perfect Hashing

Red Black Tree

1. Definition

A Red-Black Tree is a **self-balancing Binary Search Tree (BST)** where each node has a color (**red** or **black**). It ensures logarithmic height by applying balancing rules after insertion and deletion.

2. Properties of Red-Black Tree

1. Each node is either **red** or **black**.
 2. The **root is always black**.
 3. All **leaves (NIL pointers)** are considered black.
 4. If a node is red, its **children must be black** (no two consecutive reds).
 5. Every path from a node to its descendant NIL nodes has the **same number of black nodes** (called *black-height*).
-

3. Insertion in RBT

Algorithm Idea

1. Insert node like in a normal BST, color it **red**.
2. Fix violations using rotations + recoloring.

Pseudocode (Insertion Fixup)

```
RB-Insert(T, z):
    Insert z into BST as usual
    color[z] = RED
    while z ≠ root[T] and color[parent[z]] = RED:
        if parent[z] = left child of grandparent[z]:
            y = right child of grandparent[z]    // uncle
            if color[y] = RED:                    // Case 1: Uncle is RED
                color[parent[z]] = BLACK
                color[y] = BLACK
                color[grandparent[z]] = RED
                z = grandparent[z]
            else:
                if z = right child of parent[z]:  // Case 2
                    z = parent[z]
                    Left-Rotate(T, z)
                // Case 3
                color[parent[z]] = BLACK
                color[grandparent[z]] = RED
                Right-Rotate(T, grandparent[z])
        else (mirror cases for right child) ...
    color[root[T]] = BLACK
```

Example (Insert Keys: 10, 20, 30)

1. Insert 10 → root is black.
 2. Insert 20 → red child, no violation.
 3. Insert 30 → violation (two consecutive reds). Fix with rotation → Root = 20 (black), children = 10 (red), 30 (red).
-

4. Deletion in RBT

Algorithm Idea

1. Delete node like in normal BST.
2. If deleted node or replaced node was **black**, fix *double-black* violations.

Pseudocode (Deletion Fixup)

```
RB-Delete(T, z):
    if left[z] = NIL or right[z] = NIL:
        y = z
    else:
        y = Successor(z)
    if left[y] ≠ NIL:
        x = left[y]
    else:
        x = right[y]
    transplant(T, y, x)
    if y ≠ z:
        key[z] = key[y]
    if color[y] = BLACK:
        RB-Delete-Fixup(T, x)
```

```
RB-Delete-Fixup(T, x):
    while x ≠ root[T] and color[x] = BLACK:
        if x = left child of parent[x]:
            w = right sibling of x
            if color[w] = RED:                // Case 1
                color[w] = BLACK
                color[parent[x]] = RED
                Left-Rotate(T, parent[x])
                w = right sibling of x
            if color[left[w]] = BLACK and color[right[w]] = BLACK: //
Case 2
                color[w] = RED
```

```

        x = parent[x]
    else:
        if color[right[w]] = BLACK:           // Case 3
            color[left[w]] = BLACK
            color[w] = RED
            Right-Rotate(T, w)
            w = right sibling of x
        // Case 4
        color[w] = color[parent[x]]
        color[parent[x]] = BLACK
        color[right[w]] = BLACK
        Left-Rotate(T, parent[x])
        x = root[T]
    else (mirror cases) ...
color[x] = BLACK

```

Example (Delete 10 from Tree [20(B), 10(R), 30(R)])

- After deletion → restructure and recolor to maintain properties.

5. Complexity Analysis

Operation	Time Complexity	Space Complexity
Search	$O(\log n)$	$O(1)$
Insertion	$O(\log n)$	$O(1)$
Deletion	$O(\log n)$	$O(1)$

✅ Height of RBT = $O(\log n)$ because balancing ensures black-height $\leq \log(n)$.

6. Why Red-Black Tree?

- Ensures balanced BST → prevents skewed trees.
- Used in **Linux kernel**, **C++ STL map/set**, **Java TreeMap/TreeSet**.

AVL Tree

1. Definition

An **AVL Tree** is a **self-balancing Binary Search Tree** (BST) where the **balance factor** of every node is:

$BF(\text{node}) = \text{height}(\text{left}) - \text{height}(\text{right}) \in \{-1, 0, +1\}$
 $BF(\text{node}) = \text{height}(\text{left}) - \text{height}(\text{right}) \in \{-1, 0, +1\}$

If the balance factor goes outside this range, the tree is rebalanced using **rotations**.

2. Operations in AVL Tree

♦ Search

- Works the same as in BST.
- No extra rebalancing needed.

Pseudocode

```
AVL-Search(root, key):
    if root = NIL or root.key = key:
        return root
    if key < root.key:
        return AVL-Search(root.left, key)
    else:
        return AVL-Search(root.right, key)
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

◆ Insertion

1. Insert node as in normal BST.
2. Update heights and balance factors.
3. If imbalance occurs, perform one of **four rotations**.

Pseudocode

```
AVL-Insert(root, key):
    if root = NIL:
        return newNode(key)
    if key < root.key:
        root.left = AVL-Insert(root.left, key)
    else if key > root.key:
        root.right = AVL-Insert(root.right, key)

    updateHeight(root)
    balance = getBalance(root)

    // Case 1: Left-Left
    if balance > 1 and key < root.left.key:
        return Right-Rotate(root)

    // Case 2: Right-Right
    if balance < -1 and key > root.right.key:
        return Left-Rotate(root)

    // Case 3: Left-Right
    if balance > 1 and key > root.left.key:
        root.left = Left-Rotate(root.left)
        return Right-Rotate(root)

    // Case 4: Right-Left
    if balance < -1 and key < root.right.key:
```

```
        root.right = Right-Rotate(root.right)
        return Left-Rotate(root)

return root
```

♦ Deletion

1. Delete node as in normal BST.
2. Update heights.
3. If imbalance occurs, fix with rotations.

Pseudocode

```
AVL-Delete(root, key):
    if root = NIL:
        return root
    if key < root.key:
        root.left = AVL-Delete(root.left, key)
    else if key > root.key:
        root.right = AVL-Delete(root.right, key)
    else:
        if root.left = NIL or root.right = NIL:
            root = (root.left) ? root.left : root.right
        else:
            temp = minValueNode(root.right)
            root.key = temp.key
            root.right = AVL-Delete(root.right, temp.key)

    if root = NIL:
        return root

    updateHeight(root)
    balance = getBalance(root)

    // Case 1: Left-Left
    if balance > 1 and getBalance(root.left) >= 0:
```

```

        return Right-Rotate(root)

// Case 2: Left-Right
if balance > 1 and getBalance(root.left) < 0:
    root.left = Left-Rotate(root.left)
    return Right-Rotate(root)

// Case 3: Right-Right
if balance < -1 and getBalance(root.right) <= 0:
    return Left-Rotate(root)

// Case 4: Right-Left
if balance < -1 and getBalance(root.right) > 0:
    root.right = Right-Rotate(root.right)
    return Left-Rotate(root)

return root

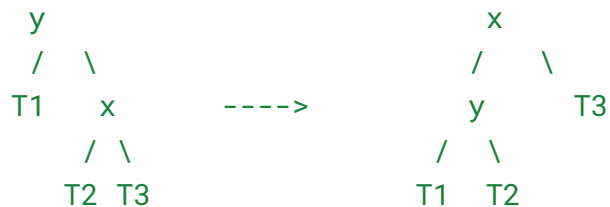
```

3. Rotations in AVL Tree

(a) Right Rotation (LL Case)



(b) Left Rotation (RR Case)



(c) Left-Right Rotation (LR Case)

- Perform **Left Rotation** on left child, then **Right Rotation** on root.

(d) Right-Left Rotation (RL Case)

- Perform **Right Rotation** on right child, then **Left Rotation** on root.
-

4. Examples

Example 1: Insert [10, 20, 30]

- Insert 10 → root.
- Insert 20 → right child.
- Insert 30 → imbalance (RR case).
- Perform **Left Rotation**.
- Final Tree: 20 (root), 10 (left), 30 (right).

Example 2: Insert [30, 10, 20]

- Insert 30 → root.
 - Insert 10 → left child.
 - Insert 20 → imbalance (LR case).
 - Perform **Left Rotation** on 10, then **Right Rotation** on 30.
 - Final Tree: 20 (root), 10 (left), 30 (right).
-

5. Complexity Analysis

Operation	Time Complexity	Space Complexity
Search	$O(\log n)$	$O(1)$
Insertion	$O(\log n)$	$O(1)$
Deletion	$O(\log n)$	$O(1)$

✓ Height of AVL Tree = $O(\log n)$.

✓ Balances more strictly than Red-Black → **faster lookups**.

6. Key Points

- AVL guarantees stricter balance than Red-Black.
- **AVL better for search-heavy applications.**
- **Red-Black better for insertion/deletion-heavy applications (less rebalancing).**

SORTING

1. Quick Sort

Idea: Divide & Conquer. Pick a pivot, partition array into two subarrays (smaller and larger than pivot), recursively sort.

Pseudocode:

QUICKSORT(A, low, high):

```

    if low < high:
        pivot = PARTITION(A, low, high)
        QUICKSORT(A, low, pivot-1)
        QUICKSORT(A, pivot+1, high)

PARTITION(A, low, high):
    x = A[high] # pivot
    i = low-1
    for j = low to high-1:
        if A[j] <= x:
            i = i+1
            swap A[i], A[j]
    swap A[i+1], A[high]
    return i+1

```

Complexity:

- Best = $O(n \log n)$
- Average = $O(n \log n)$
- Worst = $O(n^2)$ (pivot always smallest/largest)
- Space = $O(\log n)$ (recursive stack)
- Stable = ✗ No
- ASC / DESC = depends on partition logic, can sort either way

Use: Best for large datasets when average performance matters.

2. Randomized Quick Sort

Idea: Same as quick sort but choose pivot randomly → reduces chance of worst case.

Change: before partition, swap pivot with random element.

Complexity:

- Best = $O(n \log n)$
- Average = $O(n \log n)$
- Worst = $O(n^2)$ (very rare)
- Stable =  No

Use: Better than normal quick sort in practice (less chance of skewed partition).

3. Radix Sort

Idea: Sort numbers digit by digit using stable sorting (e.g., counting sort) from least significant digit (LSD) to most significant (MSD).

Pseudocode:

```
RADIXSORT(A, d):  # d = number of digits
    for i = 1 to d:
        STABLECOUNTINGSORT(A, digit=i)
```

Complexity:

- Time = $O(d \cdot (n+k))$ where k = range of digits
- Best = $O(nk)$, Average = $O(nk)$, Worst = $O(nk)$
- Space = $O(n+k)$
- Stable =  Yes

Use: Best for integers, phone numbers, strings with fixed length.

4. Merge Sort

Idea: Divide array into halves, recursively sort, then merge.

Pseudocode:

```
MERGESORT(A, l, r):  
    if l < r:  
        m = (l+r)/2  
        MERGESORT(A, l, m)  
        MERGESORT(A, m+1, r)  
        MERGE(A, l, m, r)
```

Complexity:

- Best = $O(n \log n)$
- Average = $O(n \log n)$
- Worst = $O(n \log n)$
- Space = $O(n)$ (extra array in merge)
- Stable = ☒ Yes

Use: Good for linked lists & external sorting (large files).

5. Heap Sort

Idea: Build max-heap, repeatedly extract max, place at end.

Pseudocode:

```
HEAPSORT(A):  
    BUILD-MAX-HEAP(A)  
    for i = n downto 2:  
        swap A[1], A[i]  
        heap-size = heap-size - 1  
        MAX-HEAPIFY(A, 1)
```

Complexity:

- Best = $O(n \log n)$
- Average = $O(n \log n)$
- Worst = $O(n \log n)$
- Space = $O(1)$
- Stable =  No

Use: In-place sorting, not stable but memory-efficient.

6. Bucket Sort

Idea: Distribute elements into buckets, sort each bucket (e.g., insertion sort), concatenate.

Pseudocode:

```
BUCKETSORT(A):  
    create n buckets  
    for i in A:  
        insert i into appropriate bucket  
    sort each bucket  
    concatenate all buckets
```

Complexity:

- Best = $O(n+k)$
- Average = $O(n+k)$
- Worst = $O(n^2)$ (if all go in one bucket)

- Space = $O(n+k)$
- Stable = depends on bucket sort used

Use: Uniformly distributed floating-point numbers in $[0,1)$.

7. Counting Sort

Idea: Count frequency of each element, calculate positions, place into output.

Pseudocode:

```
COUNTINGSORT(A, k):
    C[0..k] = 0
    for j in A:
        C[A[j]] = C[A[j]]+1
    for i=1..k:
        C[i] = C[i] + C[i-1]
    for j=n downto 1:
        output[C[A[j]]] = A[j]
        C[A[j]]--
```

Complexity:

- Time = $O(n+k)$
- Space = $O(n+k)$
- Stable =  Yes
- Best when $k = O(n)$.

Use: Integers in small range.

8. Bubble Sort

Idea: Repeatedly swap adjacent elements if out of order.

Pseudocode:

```
BUBBLESORT(A):  
    for i=1 to n-1:  
        for j=1 to n-i:  
            if A[j] > A[j+1]:  
                swap A[j], A[j+1]
```

Complexity:

- Best = $O(n)$ (already sorted)
- Average = $O(n^2)$
- Worst = $O(n^2)$
- Space = $O(1)$
- Stable =  Yes

Use: Only for small datasets or teaching purposes.

9. Insertion Sort

Idea: Build sorted array one item at a time.

Pseudocode:

```
INSERTIONSORT(A):  
    for i=2 to n:  
        key = A[i]  
        j = i-1  
        while j>0 and A[j]>key:  
            A[j+1]=A[j]
```

```
j=j-1  
A[j+1]=key
```

Complexity:

- Best = $O(n)$
- Average = $O(n^2)$
- Worst = $O(n^2)$
- Space = $O(1)$
- Stable = ☒ Yes

Use: Small arrays, nearly sorted data.

10. Selection Sort

Idea: Select minimum element, swap with current index, repeat.

Pseudocode:

```
SELECTIONSORT(A):  
    for i=1 to n-1:  
        min = i  
        for j=i+1 to n:  
            if A[j] < A[min]:  
                min = j  
        swap A[i], A[min]
```

Complexity:

- Best = $O(n^2)$
- Average = $O(n^2)$

- Worst = $O(n^2)$
- Space = $O(1)$
- Stable = ✗ No

Use: Simple to implement, not practical for large datasets.

Algo	Best	Avg	Worst	Space	Stable	Notes
Quick Sort	$n \log n$	$n \log n$	n^2	$\log n$	No	Fast, in-place
Randomized Quick	$n \log n$	$n \log n$	n^2	$\log n$	No	Avoids bad pivot
Merge Sort	$n \log n$	$n \log n$	$n \log n$	n	Yes	External sorting
Heap Sort	$n \log n$	$n \log n$	$n \log n$	1	No	In-place
Radix Sort	nk	nk	nk	$n+k$	Yes	Good for integers
Bucket Sort	$n+k$	$n+k$	n^2	$n+k$	Depends	Good for $[0, 1)$ floats
Counting Sort	$n+k$	$n+k$	$n+k$	$n+k$	Yes	Integers, limited range
Bubble Sort	n	n^2	n^2	1	Yes	Educational only
Insertion Sort	n	n^2	n^2	1	Yes	Small/nearly sorted data
Selection Sort	n^2	n^2	n^2	1	No	Rarely used

Binary MaxHeap/MinHeap

1. Definition

- A **Binary Heap** is a **complete binary tree** (all levels filled except possibly the last, filled left to right).
- Stored as an **array**:

- Root at index 1 (or 0 depending on implementation).
- Left child at $2*i$, Right child at $2*i + 1$.
- Parent at $i/2$.

Types

1. **MaxHeap**: Each parent $\geq$ children.
 2. **MinHeap**: Each parent $\leq$ children.
-

2. Insertion in Heap

Algorithm

1. Insert new element at the end (bottom right).
2. "Bubble up" / **Heapify-Up** until heap property is restored.

Pseudocode

```
Heap-Insert(A, key):  
    size = size + 1  
    i = size  
    A[i] = key  
    while i > 1 and A[parent(i)] < A[i]:    // For MaxHeap  
        swap(A[i], A[parent(i)])  
        i = parent(i)
```

3. Deletion in Heap (Delete Root)

Algorithm

1. Replace root with last element.
2. Remove last element.
3. "Bubble down" / **Heapify-Down** from root.

Pseudocode

Heap-Delete-Root(A):

```
    if size < 1:
        return "Heap Underflow"
    root = A[1]
    A[1] = A[size]
    size = size - 1
    Max-Heapify(A, 1)
    return root
```

Max-Heapify(A, i):

```
    l = 2*i
    r = 2*i + 1
    largest = i
    if l ≤ size and A[l] > A[largest]:
        largest = l
    if r ≤ size and A[r] > A[largest]:
        largest = r
    if largest ≠ i:
        swap(A[i], A[largest])
        Max-Heapify(A, largest)
```

4. Finding kth Max / kth Min

Method 1: Using Heap Directly

- For **kth max**:
 - Build **MaxHeap**, extract max **k** times.

- Complexity: $O(n + k \log n)$.
- For **kth min**:
 - Build **MinHeap**, extract min k times.
 - Complexity: $O(n + k \log n)$.

Pseudocode (kth max using MaxHeap)

```
Find-Kth-Max(A, k):
    Build-Max-Heap(A)
    for i = 1 to k:
        val = Heap-Delete-Root(A)
    return val
```

Method 2: Efficient using size-k heap

- To find **kth largest**:
 - Use **MinHeap of size k**.
 - Traverse array, keep inserting. If size > k, pop smallest.
 - Root of heap = kth largest.
 - Complexity: $O(n \log k)$.
- To find **kth smallest**:
 - Use **MaxHeap of size k**.

5. Examples

Example 1: MaxHeap Insert [10, 20, 5, 30]

1. Insert 10 $\rightarrow$ [10]

2. Insert 20 → bubble up → [20, 10]
3. Insert 5 → [20, 10, 5]
4. Insert 30 → bubble up → [30, 20, 5, 10]

Example 2: Delete Root (30)

- Replace with 10 → [10, 20, 5].
 - Bubble down → [20, 10, 5].
-

6. Complexity Analysis

Operation	Time Complexity	Space Complexity
Build Heap	$O(n)$	$O(1)$
Insertion	$O(\log n)$	$O(1)$
Deletion (root)	$O(\log n)$	$O(1)$
Search arbitrary key	$O(n)$	$O(1)$
Find kth max/min	$O(n + k \log n)$ or $O(n \log k)$	$O(k)$

✓ Heap guarantees **logarithmic insertion/deletion** but does **not support fast search**.

7. Key Points

- Used in **HeapSort, Priority Queues, Dijkstra's Algorithm, Prim's Algorithm**.
- **MaxHeap** → **Priority for maximum values**.
- **MinHeap** → **Priority for minimum values**.

Binary Search Tree

1. Definition

- A Binary Search Tree is a binary tree where for every node:
 - Left subtree contains keys **less than** the node's key.
 - Right subtree contains keys **greater than** the node's key.
 - Both left and right subtrees are also BSTs.
-

2. Insertion in BST

Algorithm

1. Start from root.
2. If $\text{key} < \text{root}$, go to left subtree; else go to right subtree.
3. Insert when a **NIL** is found.

Pseudocode

```
BST-Insert(root, key):  
    if root = NIL:  
        return newNode(key)  
    if key < root.key:  
        root.left = BST-Insert(root.left, key)  
    else if key > root.key:  
        root.right = BST-Insert(root.right, key)
```

```
return root
```

3. Deletion in BST

Algorithm

1. Search for the node.
2. Three cases:
 - **Case 1:** Node has no child → delete directly.
 - **Case 2:** Node has one child → replace node with child.
 - **Case 3:** Node has two children → replace with **inorder successor** (minimum in right subtree), then delete successor.

Pseudocode

```
BST-Delete(root, key):  
    if root = NIL:  
        return root  
    if key < root.key:  
        root.left = BST-Delete(root.left, key)  
    else if key > root.key:  
        root.right = BST-Delete(root.right, key)  
    else:  
        // Node found  
        if root.left = NIL:  
            return root.right  
        else if root.right = NIL:  
            return root.left  
        else:  
            temp = MinValueNode(root.right)  
            root.key = temp.key  
            root.right = BST-Delete(root.right, temp.key)  
    return root
```

```
MinValueNode(node):  
    current = node  
    while current.left != NIL:  
        current = current.left  
    return current
```

4. Check if a Tree is a BST

Method

- Use **Inorder Traversal** → should give a sorted sequence.
- Or recursively check min/max constraints.

Pseudocode

```
IsBST(node, min, max):  
    if node = NIL:  
        return true  
    if node.key ≤ min or node.key ≥ max:  
        return false  
    return IsBST(node.left, min, node.key) AND  
           IsBST(node.right, node.key, max)
```

5. Find an Element in BST

Algorithm

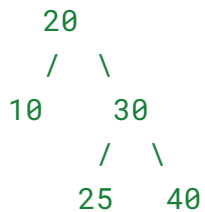
1. Start from root.
2. If key = root → found.
3. If key < root → search left.
4. If key > root → search right.

Pseudocode

```
BST-Search(root, key):  
    if root = NIL or root.key = key:  
        return root  
    if key < root.key:  
        return BST-Search(root.left, key)  
    else:  
        return BST-Search(root.right, key)
```

6. Examples

Example 1: Insert [20, 10, 30, 25, 40]



Example 2: Delete 30

- Successor = 40 → replace 30 with 40.



Example 3: Check BST

- Inorder traversal [10, 20, 25, 40] → sorted  → BST.
-

7. Complexity Analysis

Operation	Best Case (Balanced BST)	Worst Case (Skewed BST)	Space
Insertion	$O(\log n)$	$O(n)$	$O(1)$
Deletion	$O(\log n)$	$O(n)$	$O(1)$
Search	$O(\log n)$	$O(n)$	$O(1)$
Check if BST	$O(n)$	$O(n)$	$O(h)$ recursion

👉 BST performance depends on balance → skewed BST becomes like a linked list.

Fibonacci Heap

1. Definition

A **Fibonacci Heap** is a collection of **heap-ordered trees** (min-heap or max-heap).

It supports very efficient **decrease-key** and **merge (union)** operations compared to binary heaps.

- Each tree follows **min-heap property** (parent $\leq$ children).
 - Maintains a **root list** of trees, linked circularly.
 - Uses **lazy operations** → restructuring happens only when necessary.
-

2. Operations

- ♦ **Insertion**

Idea

1. Create a new node.
2. Insert it into root list.
3. Update min pointer if needed.

Pseudocode

```
Fib-Insert(H, x):  
    x.degree = 0  
    x.parent = NIL  
    x.child = NIL  
    x.mark = FALSE  
    add x to root list of H  
    if H.min = NIL or x.key < H.min.key:  
        H.min = x  
    H.n = H.n + 1
```

Time Complexity: $O(1)$ amortized

♦ Find Minimum (for Min-Heap)

Idea

- Just return pointer `H.min`.

Pseudocode

```
Fib-Find-Min(H):  
    return H.min
```

Time Complexity: $O(1)$

♦ Extract Minimum (Deletion of Min)

Idea

1. Remove `H.min`.
2. Add its children to root list.
3. Consolidate: merge roots of equal degree.

Pseudocode

```
Fib-Extract-Min(H):  
    z = H.min  
    if z ≠ NIL:  
        for each child x of z:  
            add x to root list of H  
            x.parent = NIL  
        remove z from root list  
        if root list not empty:  
            Consolidate(H)  
        else:  
            H.min = NIL  
        H.n = H.n - 1  
    return z
```

Time Complexity: $O(\log n)$ amortized

♦ Search for an Element

Fibonacci Heap does **not support fast search** directly.
Naive search requires traversing the heap.

Pseudocode

```
Fib-Search(H, key):  
    for each root r in root list:  
        if DFS(r, key):  
            return TRUE  
    return FALSE
```

```

DFS(node, key):
    if node.key == key:
        return TRUE
    for each child c of node:
        if DFS(c, key):
            return TRUE
    return FALSE

```

Time Complexity: $O(n)$

3. Complexity Analysis

Operation	Amortized Time	Worst Case
Insert	$O(1)$	$O(1)$
Find Min	$O(1)$	$O(1)$
Union (Merge)	$O(1)$	$O(1)$
Extract Min	$O(\log n)$	$O(\log n)$
Decrease Key	$O(1)$	$O(\log n)$
Delete	$O(\log n)$	$O(\log n)$
Search	$O(n)$	$O(n)$

✓ Fibonacci Heap is especially powerful in algorithms like **Dijkstra's shortest path** and **Prim's MST**, where **decrease-key** is heavily used.

4. Example

Insert [10, 3, 15, 6] into a Min Fibonacci Heap:

- Insert 10 → min = 10

- Insert 3 → min = 3
- Insert 15 → min still 3
- Insert 6 → min still 3

Extract-Min → remove 3, restructure → new min = 6

5. Key Points

- Better asymptotic performance than Binary Heap.
- Not widely used in practice because constants are high & implementation is complex.
- Theoretical importance: improves **Dijkstra's and Prim's algorithms** from $O(E \log V)$ → $O(E + V \log V)$.

Binomial Heap

1. Definition

A **Binomial Heap** is a collection of **binomial trees** that satisfies:

1. Each tree is **min-heap ordered** (key of parent $\leq$ key of children).
2. At most **one binomial tree of each degree** is present.

👉 A **binomial tree** B_k has:

- Height = k .
- Root has degree k .
- Exactly 2^k nodes.

- Children are roots of binomial trees $B_0, B_1, \dots, B_{(k-1)}$.
-

2. Operations

♦ Insertion

Idea

- Create a heap with a single node.
- Perform **Union** with the existing heap.

Pseudocode

```
Binomial-Insert(H, x):  
    H1 = Make-Heap()  
    H1.head = x  
    return Binomial-Union(H, H1)
```

Time Complexity: $O(\log n)$

♦ Find Minimum

Idea

- Traverse root list, return node with minimum key.

Pseudocode

```
Binomial-Find-Min(H):  
    min =  $\infty$   
    y = NIL  
    x = H.head  
    while x  $\neq$  NIL:  
        if x.key < min:  
            min = x.key  
            y = x  
        x = x.sibling  
    return y
```

Time Complexity: $O(\log n)$

♦ Deletion

Idea

- To delete a node x :
 1. Decrease its key to $-\infty$ (for MinHeap).
 2. Bubble it up to root.
 3. Perform **Extract-Min**.

Pseudocode

```
Binomial-Delete(H, x):  
    Binomial-Decrease-Key(H, x,  $-\infty$ )  
    Binomial-Extract-Min(H)
```

Time Complexity: $O(\log n)$

♦ Search

Idea

- Perform DFS on all binomial trees.

Pseudocode

```
Binomial-Search(H, key):  
    for each root r in root list:  
        if DFS(r, key):  
            return TRUE  
    return FALSE
```

```
DFS(node, key):  
    if node.key = key:  
        return TRUE  
    child = node.child  
    while child ≠ NIL:  
        if DFS(child, key):  
            return TRUE  
        child = child.sibling  
    return FALSE
```

Time Complexity: $O(n)$

♦ Union (Merge Heaps)

Idea

1. Merge root lists (like merging sorted lists by degree).
2. Traverse merged list: if two consecutive trees have same degree → link them.

Pseudocode

```
Binomial-Union(H1, H2):
    H = Make-Heap()
    H.head = Binomial-Merge(H1, H2)
    if H.head = NIL:
        return H
    prev = NIL
    x = H.head
    next = x.sibling
    while next ≠ NIL:
        if (x.degree ≠ next.degree) or
            (next.sibling ≠ NIL and next.sibling.degree = x.degree):
            prev = x
            x = next
        else if x.key ≤ next.key:
            x.sibling = next.sibling
            Binomial-Link(next, x)
        else:
            if prev = NIL:
                H.head = next
            else:
                prev.sibling = next
                Binomial-Link(x, next)
            x = next
        next = x.sibling
    return H

Binomial-Link(y, z):    // make y child of z
    y.parent = z
    y.sibling = z.child
    z.child = y
    z.degree = z.degree + 1
```

Time Complexity: $O(\log n)$

3. Complexity Analysis

Operation	Time Complexity
Make-Heap	$O(1)$
Insert	$O(\log n)$
Minimum	$O(\log n)$
Extract-Min	$O(\log n)$
Union	$O(\log n)$
Decrease-Key	$O(\log n)$
Delete	$O(\log n)$
Search	$O(n)$

✓ Space Complexity: $O(n)$

4. Example

Insert [10, 20, 5] into Binomial Heap

1. Insert 10 → tree B_0 .
 2. Insert 20 → Union with existing → trees: $B_0(10)$, $B_0(20)$ → link → $B_1(10 \rightarrow 20)$.
 3. Insert 5 → Union with existing → link with B_1 .
Final min = 5.
-

5. Key Points

- Binomial Heap supports **fast union ($O(\log n)$)**, unlike Binary Heap.
 - Used in **Prim's MST** and **Dijkstra's algorithm** before Fibonacci Heap was proposed.
 - **Fibonacci Heap improves** some operations, but Binomial Heap is simpler and still important for theory.
-

Search

1. Linear Search

Idea

- Scan each element of array sequentially until the target is found or list ends.
- Works on **unsorted and sorted data**.

Pseudocode

```
Linear-Search(A, n, key):  
    for i = 1 to n:  
        if A[i] == key:  
            return i    // index found  
    return -1           // not found
```

Complexity

- **Best Case:** $O(1)$ (element at first position)
- **Worst Case:** $O(n)$ (element not present or at end)
- **Average Case:** $O(n)$
- **Space Complexity:** $O(1)$

2. Binary Search

Idea

- Works only on **sorted arrays**.
- Divide search interval into halves:
 - Compare middle element with target.
 - If equal → found.
 - If smaller → search right half.
 - If larger → search left half.

Pseudocode (Iterative)

```
Binary-Search(A, n, key):  
    low = 0  
    high = n - 1  
    while low ≤ high:  
        mid = (low + high) / 2  
        if A[mid] == key:  
            return mid  
        else if A[mid] < key:  
            low = mid + 1  
        else:  
            high = mid - 1  
    return -1
```

Pseudocode (Recursive)

```
Binary-Search-Recursive(A, low, high, key):  
    if low > high:  
        return -1  
    mid = (low + high) / 2  
    if A[mid] == key:  
        return mid  
    else if A[mid] < key:  
        return Binary-Search-Recursive(A, mid+1, high, key)  
    else:  
        return Binary-Search-Recursive(A, low, mid-1, key)
```

Complexity

- **Best Case:** $O(1)$ (middle element is key)
 - **Worst Case:** $O(\log n)$
 - **Average Case:** $O(\log n)$
 - **Space Complexity:** $O(1)$ for iterative, $O(\log n)$ for recursive (stack frames)
-

3. Comparison Table

Feature	Linear Search	Binary Search
Works on sorted data?	❌ No (any data)	✅ Yes (sorted only)
Time (Best)	$O(1)$	$O(1)$
Time (Worst)	$O(n)$	$O(\log n)$
Time (Average)	$O(n)$	$O(\log n)$

Space	$O(1)$	$O(1)$ iterative, $O(\log n)$ recursive
Approach	Sequential	Divide & Conquer

Sparse Matrix

◆ 1. Properties of Sparse Matrix

- A **sparse matrix** is a matrix with a **large number of zero elements** compared to non-zero elements.
- **Density** = (No. of non-zero elements) $\div$ (Total elements).
 - If density $\ll 0.5 \rightarrow$ matrix is sparse.

Example:

```
[ 0 0 5 0 ]  
[ 0 0 0 0 ]  
[ 2 0 0 0 ]  
[ 0 0 0 3 ]
```

- $\rightarrow$ 3 non-zero elements out of 16 $\rightarrow$ sparse.
-

◆ 2. Applications

- **Scientific computing** (solving large systems of equations).
- **Graph representation** (adjacency matrix for sparse graphs).
- **Image processing** (black-white images with large uniform regions).
- **Machine learning / NLP** (storing word embeddings, document-term matrices).
- **Geographical maps** (storing sparse connections).

◆ 3. Representations of Sparse Matrix

(a) 3-Tuple Representation (Array of Triplets)

- Store only non-zero elements as: [row, col, value].

Example:

Original 4x4 Matrix:

```
[ 0 0 5 0 ]
[ 0 0 0 0 ]
[ 2 0 0 0 ]
[ 0 0 0 3 ]
```

3-Tuple form:

```
[row col val]
[4  4  3]    // matrix size + count of non-zero
[0  2  5]
[2  0  2]
[3  3  3]
```

- **Space Complexity:** $O(nz)$ (nz = number of non-zero elements).
- **Time Complexity:**
 - Access: $O(nz)$ (linear search in triplet list).
 - Traversal: $O(nz)$.

(b) Linked List Representation

Each non-zero element is represented by a **node** with fields:

row, col, value, next

-

- Nodes are linked row-wise or column-wise.
- Efficient for insertion/deletion of elements.
- **Space Complexity:** $O(nz)$.
- **Time Complexity:**
 - Traversal: $O(nz)$.
 - Insertion/Deletion: $O(1)$ (if pointer given).

(c) Compressed Sparse Row (CSR) Representation (frequently asked in exams)

- Uses **3 arrays**:
 - `values[ ]` → stores non-zero elements.
 - `col_index[ ]` → stores column indices of each non-zero element.
 - `row_ptr[ ]` → stores starting index of each row in `values[ ]`.

Example:

```
Matrix:
[ 0 0 5 0 ]
[ 0 0 0 0 ]
[ 2 0 0 0 ]
[ 0 0 0 3 ]
```

```
values[]    = [5, 2, 3]
col_index[] = [2, 0, 3]
row_ptr[]   = [0, 1, 1, 2, 3]  // cumulative count per row
```

-
- **Space Complexity:** $O(nz + n + 1)$

- **Time Complexity:**
 - Access row elements: $O(\text{row size})$.
 - Full traversal: $O(nz)$.
-

◆ 4. Time Complexity Comparison

Representation	Space Complexity	Access Time	Traversal Time
3-Tuple Array	$O(nz)$	$O(nz)$	$O(nz)$
Linked List	$O(nz)$	$O(nz)$	$O(nz)$
CSR (Compressed Row)	$O(nz + n)$	$O(\text{row size})$	$O(nz)$

Divide and Conquer

1. What is Divide and Conquer?

- A **paradigm** for designing algorithms.
 - Problem is solved in 3 steps:
 1. **Divide** → Break problem into smaller subproblems.
 2. **Conquer** → Solve subproblems (recursively).
 3. **Combine** → Merge solutions of subproblems into a final solution.
-

◆ 2. General Algorithm (Pseudocode)

```
Divide-and-Conquer(P):  
    if (P is small enough):  
        Solve P directly  
    else:  
        Divide P into subproblems P1, P2, ..., Pk  
        for each Pi:  
            Solve Pi recursively  
        Combine solutions of all Pi  
        return solution
```

◆ 3. Applications of Divide and Conquer

(a) Sorting Algorithms

- **Merge Sort** → Divide array into halves, recursively sort, merge.
- **Quick Sort** → Partition array, recursively sort left and right subarrays.

(b) Searching

- **Binary Search** → Repeatedly divide sorted array and search half.

(c) Matrix Operations

- **Strassen's Matrix Multiplication** → Divide matrices into submatrices, reduce multiplications.

(d) Computational Geometry

- **Closest Pair of Points** → Divide plane into halves, solve recursively.
- **Convex Hull (Divide and Merge approach).**

(e) Numerical Problems

- **Karatsuba's Algorithm for Fast Multiplication** → Splits large numbers for efficient multiplication.
- **Exponentiation by Squaring** → Divide power problem into halves.

(f) Other Classic Uses

- Tower of Hanoi (recursive divide problem).
 - Fibonacci sequence (naïve recursive method, though not efficient).
-

◆ 4. Complexity

- Expressed using **Recurrence Relations**.
- Example:
 - Merge Sort → $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$.
 - Binary Search → $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$.

Master Theorem for Divide-and-Conquer Recurrences

The Master Theorem provides a way to determine the **time complexity** of recurrence relations of the form:

$$T(n) = a * T(n/b) + f(n)$$

Where:

- n = size of the problem
- $a \geq 1$ = number of subproblems

- $b > 1$ = factor by which the subproblem size is reduced
- $f(n)$ = cost of dividing the problem and combining results

Hashing

What is Hashing?

- A technique to **map keys to positions (indices) in a hash table** using a **hash function**.
- Used for **fast insertion, deletion, and searching**.
- Example: Store roll numbers of students in a hash table.

Hash Function (h):

$h(\text{key}) = \text{key} \bmod m$ // m = size of hash table

Problems in Hashing:

- **Collisions** → two keys hash to the same index.
- **Collision Handling:**
 - **Chaining** (linked lists).
 - **Open Addressing** (linear probing, quadratic probing, double hashing).

Time Complexity:

- Best / Average case (successful search): $O(1)$
 - Worst case (many collisions): $O(n)$
-

◆ 2. Universal Hashing

- A **randomized hashing scheme** to reduce chances of collision.
- Instead of a fixed function, we choose **h from a family of hash functions (H)**.
- Each key is equally likely to be mapped to any slot.

Definition:

A family **H** of hash functions is *universal* if:

For any two distinct keys **x** and **y**,

$$P(h(x) = h(y)) \leq 1/m \quad // \ m = \text{number of slots}$$

Example Hash Function Family:

$$h(x) = ((ax + b) \bmod p) \bmod m$$

- p = large prime $>$ universe size
- a, b chosen randomly with $a \neq 0$

Applications:

- Cryptography (randomized lookups).
- Hash tables in competitive programming.
- Prevents **worst-case attacks** (like malicious input causing collisions).

Complexity:

- Average case: $O(1)$ expected time for search, insert, delete.

◆ 3. Perfect Hashing

- A **hashing method with NO collisions**.
- Used when the **set of keys is fixed** (static dictionary).

Idea:

- Two-level hashing:
 1. First hash function distributes keys into buckets.
 2. Second-level hash function ensures no collisions inside each bucket.

Example:

- Keys = {10, 22, 37, 40}
- Hash1: $h_1(x) = x \bmod 4 \rightarrow$ distributes into buckets.
- For each bucket, construct a perfect secondary hash.

Applications:

- Compilers (reserved keywords lookup).
- Network routing tables.
- Databases (indexing with fixed key sets).

Complexity:

- Search: $O(1)$ guaranteed (not just expected).
 - Space: $O(n)$ (sometimes slightly more due to second-level hashing).
-

◆ 4. Comparison

Feature	Normal Hashing	Universal Hashing	Perfect Hashing
Collisions	Possible	Very unlikely (randomized)	None (collision-free)
Key set	Dynamic / unknown	Dynamic	Static (fixed set)
Search Time	$O(1)$ average, $O(n)$ worst	$O(1)$ expected	$O(1)$ guaranteed
Use Case	General hash tables	Security, randomized DS	Compiler keywords, DB

Self adjusting - splay tree

Splay Tree (Self-Adjusting BST)

A **Splay Tree** is a type of **self-adjusting binary search tree (BST)** where recently accessed elements are moved closer to the root to optimize future accesses.

Key Properties

1. **Self-Adjusting:** Every access (search, insert, delete) moves the accessed node to the root using **splaying**.
 2. **No Explicit Balance Information:** Unlike AVL or Red-Black trees, Splay Trees do not store height or color.
 3. **Amortized Efficiency:** Operations are efficient **on average**, with amortized time **$O(\log n)$** .
 4. **BST Property Maintained:** Left child < parent < right child.
-

Main Operations

1. Splay Operation

Moves a node x to the root using rotations. It has three types of steps:

a. Zig (Single Rotation)

- Node x is child of root.
- Perform **one rotation** to make x the root.

b. Zig-Zig (Double Rotation)

- Node x and its parent p are both **left children** or **both right children**.
- Perform **two rotations**: first on p , then on x .

c. Zig-Zag (Double Rotation)

- Node x is **left child of right parent** or **right child of left parent**.
 - Perform **two rotations**: first on x , then again on x .
-

2. Search(x)

- Search for element x .
- If found, **splay x to the root**.
- If not found, **splay the last accessed node** to the root.

Time Complexity: Amortized $O(\log n)$

3. Insertion(x)

1. Insert x like in a BST.
2. Splay x to the root.

Time Complexity: Amortized $O(\log n)$

4. Deletion(x)

1. Splay x to the root.
2. Remove the root.
3. If left subtree exists, splay the **maximum node of left subtree** to root, then attach right subtree.

Time Complexity: Amortized $O(\log n)$

5. Join(T_1, T_2)

- Combine two splay trees T_1 and T_2 where all keys in $T_1 < T_2$.
- Splay **maximum node of T_1** to root.
- Attach T_2 as right subtree of root.

Time Complexity: $O(\log n)$

6. Split(T, x)

- Split tree T into T_1 (keys $\leq x$) and T_2 (keys $> x$).
- Splay x to root.
- Left subtree $\rightarrow T_1$, Right subtree $\rightarrow T_2$.

Time Complexity: $O(\log n)$

Advantages

- Frequently accessed nodes are **quick to access**.
 - Simple **no extra memory** needed for balancing info.
 - Performs well in **amortized sense**.
-

Disadvantages

- Worst-case time per operation: **$O(n)$** (like a skewed BST).
 - Not ideal if **worst-case guarantees** are required.
-

Applications

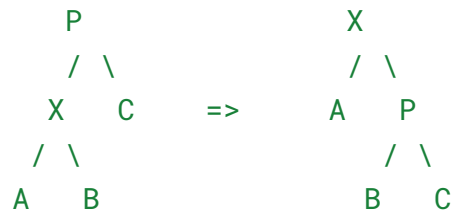
1. **Caching / Memory Management:** Move frequently used elements to root.
 2. **Networking:** Dynamic data structures for routing tables.
 3. **Amortized Data Structures:** Like **Deque**, **Priority Queues**, etc.
-

Time Complexity Summary

Operation	Worst Case	Amortized
Search	$O(n)$	$O(\log n)$
Insert	$O(n)$	$O(\log n)$
Delete	$O(n)$	$O(\log n)$
Join	$O(n)$	$O(\log n)$
Split	$O(n)$	$O(\log n)$

Splay Tree Rotations

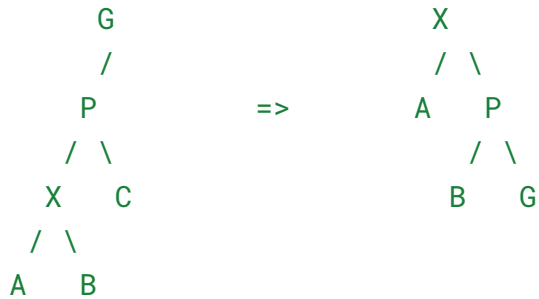
1. Zig Rotation (Single Rotation)



- X is left child of root P → single right rotation.
 - Works similarly if X is right child → left rotation.
-

2. Zig-Zig Rotation (Double Rotation)

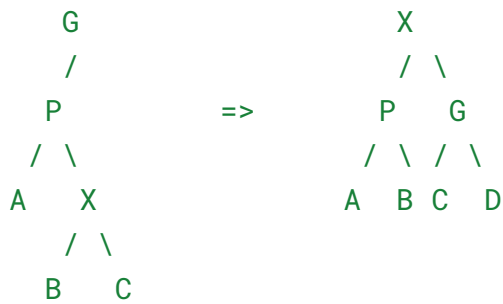
Left-Left Case



- X and P are both left children → rotate P first, then X.
 - Symmetric for right-right case (mirror image).
-

3. Zig-Zag Rotation (Double Rotation)

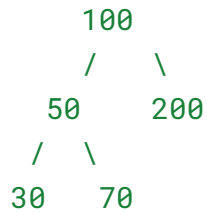
Left-Right Case



- X is left child of right parent → rotate X first on parent, then X again on grandparent.
 - Mirror image for right-left case.
-

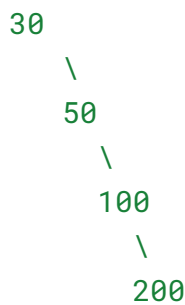
Example: Splay Operation

Initial BST:



- Search for 30 → splay 30 to root.
- Step 1: Zig-Zig rotation (30 → 50 → 100)

After splay:



- Node 30 becomes root, recently accessed node is now **quick to access next time**.

Notes

1. Remember **Zig** = one rotation, **Zig-Zig/Zig-Zag** = two rotations.
2. Splay tree **amortized $O(\log n)$** for all operations.
3. Useful in **frequently accessed data** scenarios.

End Sem

Graph

Q1 — Graph Representations (Adjacency Matrix & Adjacency List)

5-Mark Detailed Answer

Theory

A graph $G(V, E)$ consists of:

- V = set of vertices
- E = set of edges connecting vertices

To store a graph in memory, two standard representations exist:

1. **Adjacency Matrix**
 2. **Adjacency List**
-

1. Adjacency Matrix ($V \times V$ matrix)

A 2D matrix where each cell (i, j) indicates if an edge exists.

For unweighted graphs:

1 → edge exists

0 → no edge

For weighted graphs:

Matrix stores the **weight** instead of 1.

Diagram Example

Graph:

```
A --- B
|   /
|   /
C
```

Let ordering be: A=0, B=1, C=2

Adjacency Matrix:

	A	B	C
A	0	1	1
B	1	0	1
C	1	1	0

Pseudocode (Matrix Initialization)

```
Initialize matrix M[V][V] with all zeros
for each (u, v) in edges:
    M[u][v] = 1
    if undirected:
        M[v][u] = 1
```

Advantages

- Constant-time edge lookup: $O(1)$
- Easy to implement
- Good for dense graphs

Disadvantages

- Requires $O(V^2)$ space (very large for sparse graphs)
 - Iterating neighbors takes $O(V)$ time
-

Use-Cases

- Graphs with very large number of edges (dense)
 - Problems requiring fast edge existence checking (e.g., Floyd-Warshall)
-

2. Adjacency List

Stores, for each vertex, a **linked list of its neighbors**.

Diagram

Graph:

A: B, C
B: A, C
C: A, B

Pseudocode

```
Adj = array of empty lists
for each (u, v) in edges:
    Adj[u].append(v)
    if undirected:
        Adj[v].append(u)
```

Advantages

- Space efficient $\rightarrow O(V + E)$
- Fast neighbor traversal $\rightarrow O(\deg(v))$

Disadvantages

- Checking if edge exists $\rightarrow O(\deg(v))$
 - More complex than matrix
-

Use-Cases

- Sparse graphs
 - Algorithms based on neighbor traversal (BFS, DFS, Dijkstra)
-

Complexity Comparison

Operation	Matrix	List
Space	$O(V^2)$	$O(V + E)$
Check edge	$O(1)$	$O(\deg(v))$

Iterate neighbors $O(V)$ $O(\deg(v))$

Best use-case Dense Sparse



Q2 — Breadth-First Search (BFS)

5-Mark Detailed Answer

Theory

BFS is a graph traversal technique that explores nodes **level by level** from a source node.

Uses a **queue**.

Guaranteed to find the **shortest path** in **unweighted graphs**.

Diagram Example

Graph:

```
0 -- 1 -- 2
|       |
4 -- 3 -- 5
```

Step-By-Step BFS from 0

Step	Queue	Visited
1	0	0
2	1,4	0,1,4
3	4,2	0,1,4,2

4	2,3	0,1,4,2,3
5	3,5	0,1,4,2,3, 5

BFS Order:

👉 $0 \rightarrow 1 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 5$

Pseudocode

```
BFS(G, source):  
    mark all nodes unvisited  
    create empty queue Q  
    mark source visited  
    enqueue(source)  
  
    while Q not empty:  
        u = dequeue(Q)  
        for each v in Adj[u]:  
            if v not visited:  
                visited[v] = true  
                enqueue(v)
```

Complexity

- Time: $O(V + E)$
 - Space: $O(V)$
-

Use-Cases

- Shortest path in unweighted graph

- Finding connected components
 - Level-order traversal
 - Checking if a graph is bipartite
 - Solving puzzles like “minimum moves”
-
-

Q3 — Depth-First Search (DFS)

5-Mark Detailed Answer

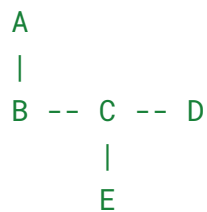
Theory

DFS explores the graph **as deep as possible** before backtracking.
Uses **recursion** or an explicit **stack**.

DFS generates:

- Discovery/finish time
 - DFS tree
 - Can find cycles
 - Used in SCC, topological sort, articulation points, bridges
-

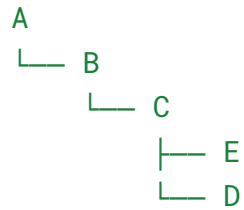
Diagram



DFS from A

Order: **A** → **B** → **C** → **E** → **D**

DFS Tree



Pseudocode

```
DFS(u):  
    mark u as visited  
    for each v in Adj[u]:  
        if not visited[v]:  
            DFS(v)
```

Complexity

- Time: **$O(V + E)$**
- Space: **$O(V)$** recursion or stack

Use-Cases

- Detecting cycles
- Topological sorting (DAG)

- Connected components
 - Path finding
 - SCC (Tarjan, Kosaraju)
 - Finding articulation points / bridges
-
-

✓ Q4 — Detecting Cycle in Undirected & Directed Graph

5-Mark Detailed Answer

A. Cycle Detection in Undirected Graph

Theory

During DFS, if we find a neighbor that is:

- **Visited**
 - **NOT** equal to the parent node
→ A cycle exists.
-

Diagram

```
1 -- 2 -- 3
|       |
4 -----
```

Clearly contains a cycle.

Pseudocode (Undirected)

```
DFS(u, parent):  
    mark u visited  
    for v in Adj[u]:  
        if v not visited:  
            if DFS(v, u) == true:  
                return true  
        else if v != parent:  
            return true
```

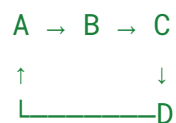
B. Cycle Detection in Directed Graph

Theory

Use **DFS + recursion stack**.

If you visit a node that is already in the **recursion stack**, then a cycle exists.

Diagram



Cycle: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

Pseudocode (Directed)

```
DFS(u):
```

```
mark u visited
mark u in recursion_stack
for v in Adj[u]:
    if not visited[v]:
        if DFS(v): return true
    else if v in recursion_stack:
        return true
remove u from recursion_stack
```

Complexity

- Time: $O(V + E)$
 - Space: $O(V)$ recursion + tracking stack
-

Use-Cases

- Deadlock detection
 - Task scheduling verification
 - Checking if topological sort possible
 - Detecting loops in dependency graphs
-
-



Q5 — Bipartite Graph & Bipartite Checking

5-Mark Detailed Answer

Theory

A graph is bipartite if:

- Vertices can be divided into **two disjoint sets** (U and V)
- Every edge connects **U** $\rightarrow$ **V** only
- No edge connects nodes within same set

Equivalent condition:

A graph is bipartite if and only if **it has no odd cycle**.

Method: BFS Coloring

Color the source node as **Red**, its neighbors **Blue**, and so on.

If any two adjacent vertices have the **same color**, graph is **not bipartite**.

Diagram

Graph that **is** bipartite:

U: A, C

V: B, D

A -- B

| |

C -- D

Graph that **is not** bipartite:

A -- B -- C -- A

(odd cycle)

Pseudocode

```
CheckBipartite(G):  
    for each vertex v:  
        color[v] = UNCOLORED  
  
    for each vertex s:  
        if color[s] == UNCOLORED:  
            BFS(s)  
            color[s] = RED  
            for each neighbor v:  
                if color[v] == UNCOLORED:  
                    color[v] = opposite(color[s])  
                else if color[v] == color[s]:  
                    return false  
  
    return true
```

Complexity

- Time: $O(V + E)$ (BFS)
 - Space: $O(V)$
-

Use-Cases

- Matching problems (maximum bipartite matching)
- Scheduling
- Map coloring
- Network flow (Ford-Fulkerson on bipartite graphs)

Q6 — Topological Sorting (DAG ordering)

Theory (what & why)

A *topological order* of a directed acyclic graph (DAG) is a linear ordering of vertices such that for every directed edge $u \rightarrow v$, u appears before v . Topological sorting is possible **iff** the graph has no directed cycles. It's used when tasks have dependencies (build systems, task scheduling, course prerequisites).

Key properties

- Valid only for DAGs.
- Not unique — many valid orders can exist.
- Detects cycles: if a topological ordering is impossible, the graph contains a cycle.

Pseudocode — Kahn's Algorithm (indegree method)

```
KahnTopologicalSort(G):
    indegree[v] = 0 for all v in V
    for each edge u->v in E:
        indegree[v]++

    Q = queue of all v with indegree[v] == 0
    topo_order = empty list

    while Q not empty:
        u = dequeue(Q)
        append u to topo_order
        for each v in Adj[u]:
            indegree[v]--
            if indegree[v] == 0:
                enqueue(Q, v)

    if |topo_order| < |V|:
        return "Graph has a cycle"    # no topo order
    else:
        return topo_order
```

Alternate pseudocode (DFS method)

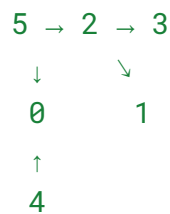
Run DFS; on finishing (post-order) push node to stack. Pop stack to get topo order.

Complexity

- Time: $O(V + E)$ (both Kahn and DFS-based)
- Space: $O(V)$ for indegree / recursion stack.

Example + step-by-step

Graph (DAG):



Edges: $5 \rightarrow 2$, $5 \rightarrow 0$, $4 \rightarrow 0$, $2 \rightarrow 3$, $2 \rightarrow 1$

Compute indegrees:

0: 2 (from 5 and 4)
1: 1 (from 2)
2: 1 (from 5)
3: 1 (from 2)
4: 0
5: 0

Kahn process:

1. $Q = [4, 5]$ (any order). Pick 4 $\rightarrow$ topo = [4]. Reduce indegree(0): $2 \rightarrow 1$.
2. $Q = [5]$. Pick 5 $\rightarrow$ topo = [4, 5]. Reduce indegree(2): $1 \rightarrow 0 \rightarrow$ enqueue 2. Reduce indegree(0): $1 \rightarrow 0 \rightarrow$ enqueue 0.
3. $Q = [2, 0]$. Pick 2 $\rightarrow$ topo = [4, 5, 2]. Reduce indegree(3): $1 \rightarrow 0$ enqueue 3. Reduce indegree(1): $1 \rightarrow 0$ enqueue 1.

4. Continue → possible topo order: `[4, 5, 2, 0, 3, 1]`.

Use-cases

- Job scheduling with prerequisites (compilers, makefiles)
 - Course prerequisite planning
 - Resolving symbol dependencies in linkers
-

Q7 — Strongly Connected Components (Kosaraju's Algorithm)

Theory (what & why)

In a directed graph, a *strongly connected component* (SCC) is a maximal set of vertices where every vertex is reachable from every other vertex in the set. SCCs partition vertices of a directed graph. Finding SCCs helps compress a graph into a DAG of components (component graph), used in program analysis, SCC condensation, and optimization.

Kosaraju's algorithm — high-level idea

1. Run a DFS on the original graph and push vertices onto a stack in order of **finish time** (post-order).
2. Reverse all edges (transpose graph).
3. Pop vertices from the stack; for each popped vertex not yet assigned, run DFS on the transposed graph — nodes reached belong to one SCC.

Pseudocode

```
Kosaraju(G):  
    stack = empty  
    visited = set()  
  
    # 1. Fill order by finish time  
    for each v in V:
```

```

        if v not visited:
            DFS1(v)

# 2. Transpose G: G_rev (reverse all edges)

# 3. Process vertices in decreasing finish time
visited = set()
while stack not empty:
    v = pop(stack)
    if v not visited:
        component = []
        DFS2(v, component)    # on G_rev
        output component

```

Complexity

- Time: $O(V + E)$ (two DFS passes + edge reversal)
- Space: $O(V + E)$ for graph and recursion/stack.

Example + diagram

Original graph:

```

1 → 2 → 3 → 1
3 → 4
5 → 6 → 5
4 → 5

```

SCCs:

- $\{1, 2, 3\}$ (cycle among 1,2,3)
- $\{4\}$ alone if no back edge to 3
- $\{5, 6\}$ (cycle between 5 and 6)

Step sketch

- DFS order (finish times) might push: 1,2,3 finish last → stack: [4,3,2,1,6,5] (example)
- Reverse edges and pop — DFS from top will reveal SCCs.

Use-cases

- Identify cyclic modules in software dependency graphs
- Reduce graph for DAG-based dynamic programming on components
- Detect mutually dependent packages/classes

Q8 — Planarity & Kuratowski's Theorem

Theory (what & why)

A graph is *planar* if it can be drawn in the plane without edge crossings. Planarity is a fundamental structural property used in graph drawing, circuit layout (VLSI), geographic mapping, and network visualization.

Important facts

- For a connected planar simple graph with $V \geq 3$ and E edges and F faces: Euler's formula:

$$V - E + F = 2.$$
- From Euler's formula derive necessary inequality: For planar simple graphs, $E \leq 3V - 6$ (for $V \geq 3$). For triangle-free planar graphs: $E \leq 2V - 4$.

Kuratowski's Theorem (characterization)

A finite graph is non-planar **iff** it contains a subgraph that is a subdivision (homeomorphic copy) of K_5 (complete graph on 5 vertices) or $K_{3,3}$ (complete bipartite with partitions of size 3).

Informally: if you can find K_5 or $K_{3,3}$ inside the graph (possibly after splitting edges into chains), the graph is non-planar.

Examples + diagrams (ASCII)

K_5 (non-planar):

```

1
 /|\
2-+-3
  \|/
  4
  5 (imagine edges fully connected - impossible planar)

```

$K_{3,3}$ (utility graph — non-planar):

```

a   b   c
| \ | / |
x   y   z   (a,b,c connected to x,y,z fully)

```

You can't draw $K_{3,3}$ in plane without crossings.

Planarity test algorithms

- Linear-time algorithms exist (Hopcroft–Tarjan), but they're complex.
- Quick heuristics: use Kuratowski motivated subgraph search or apply $E \leq 3V - 6$ as quick rejection for dense graphs.

Use-cases

- PCB and chip layout (minimize crossovers)
- Geographic graph drawing (roads, utility networks)
- Graph drawing and visualization

Complexity

- Checking planarity can be done in $O(V)$ time with advanced algorithms; simpler methods may take more and are more involved conceptually.
-

Q9 — Dijkstra's Algorithm (Single-Source Shortest Paths, non-negative weights)

Theory (what & why)

Dijkstra's algorithm computes shortest path distances from a source s to all other vertices in a graph with **non-negative** edge weights. It is greedy: it repeatedly finalizes the currently closest unsettled vertex.

Important constraints

- Edge weights must be ≥ 0 . Negative edges break correctness.
- When implemented with a binary heap (priority queue) and adjacency list, it's efficient for sparse graphs.

Pseudocode (standard, with min-priority queue)

```
Dijkstra(G, source):
    for each v in V:
        dist[v] =  $\infty$ 
        parent[v] = NIL
    dist[source] = 0
    PQ = min-priority-queue of (dist[v], v)

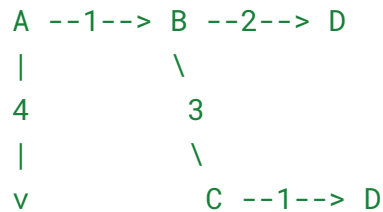
    while PQ not empty:
        (d,u) = extract-min(PQ)      # vertex with smallest dist
        if d > dist[u]: continue    # outdated entry (if decrease-key
not supported)
        for each (u,v, w) in Adj[u]:
            if dist[v] > dist[u] + w:
                dist[v] = dist[u] + w
                parent[v] = u
                decrease-key(PQ, v, dist[v])
    return dist, parent
```

Complexity

- Using binary heap / priority queue: $O((V + E) \log V)$ which is $O(E \log V)$ for connected sparse graphs.
- Using Fibonacci heap: $O(E + V \log V)$ (theoretically faster but complex).
- Using adjacency matrix (dense graph): $O(V^2)$.

Worked example (step-by-step)

Graph:



Edges: $A \rightarrow B$ (1), $A \rightarrow C$ (4), $B \rightarrow D$ (2), $B \rightarrow C$ (3), $C \rightarrow D$ (1)

Initialize:

$\text{dist}(A)=0, \text{dist}(B)=\infty, \text{dist}(C)=\infty, \text{dist}(D)=\infty$
 $PQ = [(\theta, A)]$

Process:

1. Extract A (0). Relax edges:
 - B: $\text{dist} = 0+1 = 1$ (update)
 - C: $\text{dist} = 0+4 = 4$ (update)
PQ contains (1,B),(4,C)
2. Extract B (1). Relax:
 - D: $\text{dist} = 1+2 = 3$ (update)
 - C: potential $1+3=4$ equals existing 4 $\rightarrow$ no change
PQ: (3,D),(4,C)

3. Extract D (3). Relax: maybe no outgoing or no improvement.
4. Extract C (4). Relax $C \rightarrow D$ gives $4+1=5 > \text{current } 3 \rightarrow \text{no change}$.

Final distances: $A:0$, $B:1$, $C:4$, $D:3$. Shortest path $A \rightarrow D$ is $A \rightarrow B \rightarrow D$ cost 3.

Use-cases

- Routing (OSPF in networks uses Dijkstra)
 - Map navigation (shortest driving route without negative roads)
 - Any single-source shortest path problem with non-negative weights
-

Q10 — Bellman–Ford Algorithm (Single-Source with negative edges & cycle detect)

Theory (what & why)

Bellman–Ford computes shortest paths from a single source in graphs that may contain **negative-weight edges** (but no negative cycles reachable from the source). It also **detects negative cycles** which make shortest paths undefined (arbitrarily low).

Key idea

Relax all edges repeatedly. After at most $V - 1$ iterations, all shortest path distances (with at most $V - 1$ edges) will be found. A further iteration that improves any distance indicates a negative cycle.

Pseudocode

```
BellmanFord(G, source):  
    for each v in V:  
        dist[v] =  $\infty$   
        parent[v] = NIL  
    dist[source] = 0
```

```

for i = 1 to V-1:
    for each edge (u, v, w) in E:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            parent[v] = u

# Check for negative-weight cycles
for each edge (u, v, w) in E:
    if dist[u] + w < dist[v]:
        return "Negative-weight cycle detected"

return dist, parent

```

Complexity

- Time: $O(V \times E)$ — slower than Dijkstra for non-negative weights.
- Space: $O(V)$.

Worked example

Graph:

```

S --4--> A --(-3)--> B
S --5--> B

```

Edges: $S \rightarrow A$ (4), $A \rightarrow B$ (-3), $S \rightarrow B$ (5)

Initialize:

```
dist(S)=0, dist(A)=∞, dist(B)=∞
```

Iteration 1:

- Relax $S \rightarrow A$: $\text{dist}(A)=4$
- Relax $A \rightarrow B$: $\text{dist}(B)=4 + (-3) = 1$

- Relax $S \rightarrow B$: $\text{dist}(B)$ currently $1 < 5 \rightarrow$ no change

No further improvements would occur: distances $S:0$, $A:4$, $B:1$.

If we had a cycle with negative total weight reachable from S , a final pass would decrease some $\text{dist}[]$, signaling a negative cycle.

Use-cases

- When negative edge weights model e.g., profit/loss or constraints (currency arbitrage detection)
- As a subroutine in algorithms needing negative-edge support
- Detect negative cycles in financial networks or constraint graphs

When to prefer Bellman–Ford vs Dijkstra

- If graph has negative edges $\rightarrow$ use Bellman–Ford.
- If all edges $\geq 0 \rightarrow$ Dijkstra is faster and preferred.

1) Floyd–Warshall Algorithm – All-Pairs Shortest Path

Theory

The Floyd–Warshall algorithm is a **dynamic programming** algorithm that finds **shortest paths between every pair of vertices** in a weighted graph.

It works for:

- ✓ Directed / Undirected graphs
- ✓ Positive + Negative weights
- ✗ But **no negative cycles**

It is based on the idea:

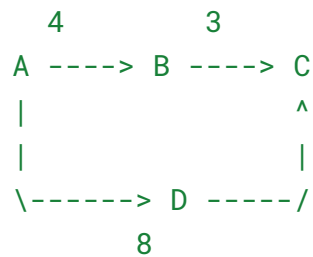
“Try to improve the shortest path between every pair (i, j) using an intermediate vertex k.”

DP Recurrence

$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$
 $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$

Graph Example

Graph:



Step-by-Step Table Example

Initial distance matrix:

	A	B	C	D
A	0	4	∞	8
B	∞	0	3	∞
C	∞	∞	0	∞
D	∞	∞	2	0

Then try $k = A, B, C, D$, and keep updating.

Final matrix contains shortest distances between every pair.

Pseudocode

```

for k in V:
    for i in V:
        for j in V:
            if dist[i][j] > dist[i][k] + dist[k][j]:
                dist[i][j] = dist[i][k] + dist[k][j]
  
```

Use Cases

- Routing tables in computer networks
- GPS / transit distance calculation
- Finding transitive closure
- Detecting negative cycles

Complexity

- **Time:** $O(V^3)$
 - **Space:** $O(V^2)$
-

2) Topological Sorting (DAG Only)

Theory

Topological Sorting is a linear ordering of vertices such that:

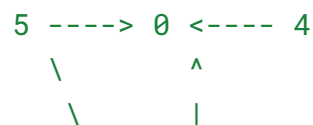
“For every directed edge $u \rightarrow v$, u appears before v in the ordering.”

It is ONLY possible for **Directed Acyclic Graphs (DAGs)**.

Two approaches:

1. **Kahn's Algorithm (BFS + In-Degree Method)**
2. **DFS Finishing Time Ordering**

Example Graph



v |
2 -> 3 -> 1

Possible topological ordering:

5, 4, 2, 3, 1, 0

Kahn's Algorithm – Steps

1. Compute **in-degree** of all vertices
2. Push all vertices with **in-degree 0** into queue
3. Pop element → add to ordering
4. Reduce in-degree of its neighbors
5. If any neighbor becomes 0 → push to queue
6. Repeat

Pseudocode

```
queue = all vertices with indegree = 0
while queue not empty:
    u = queue.pop()
    topo.push(u)

    for each v in adj[u]:
        indegree[v]--
        if indegree[v] == 0:
            queue.push(v)
```

Use Cases

- Task scheduling
- Build system dependencies
- Course prerequisite ordering

- Resolving package dependencies

Complexity

- **Time:** $O(V + E)$
 - **Space:** $O(V)$
-

3) Cycle Detection in Directed Graph (Using DFS + Recursion Stack)

Theory

A directed cycle exists if you can start at a vertex and return back to it following directed edges.
We use DFS with two arrays:

- **visited[v]** → node has been visited
- **recStack[v]** → node is currently in DFS path

If we find an edge:

$u \rightarrow v$ where `recStack[v] = true`,
then a **cycle is found**.

Example

Graph:

```
A → B → C
↑         |
|-----|
```

This graph contains a cycle: $A \rightarrow B \rightarrow C \rightarrow A$

Pseudocode

```
DFS(u):
    visited[u] = true
    recStack[u] = true

    for v in adj[u]:
        if not visited[v] and DFS(v):
            return true
        elif recStack[v]:
            return true

    recStack[u] = false
    return false
```

Use Cases

- Checking if a graph is a DAG (for topological sort)
- Detecting deadlocks in OS
- Circular dependency detection

Complexity

- **Time:** $O(V + E)$
 - **Space:** $O(V)$
-

4) Cycle Detection in Undirected Graph (DFS)

Theory

In an undirected graph, if during DFS we reach a **visited vertex that is not the parent**, then a cycle exists.

Example

```
1 -- 2
|  /  |
|  /  |
3 -- 4
```

Cycle: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$

DFS Rule

- If v is visited and $v \neq \text{parent}$, then **cycle exists**

Pseudocode

```
DFS(u, parent):
    visited[u] = true
    for v in adj[u]:
        if not visited[v]:
            if DFS(v, u):
                return true
        elif v != parent:
            return true
    return false
```

Use Cases

- Social network clustering
- Graph validation problems
- Network topology cycle detection

Complexity

- **Time:** $O(V + E)$
- **Space:** $O(V)$

5) Articulation Points (Cut Vertices)

Theory

An articulation point (AP) is a vertex whose removal **increases the number of connected components** of the graph.

We use **DFS + discovery time + low time**:

- **disc[u]** → time when u is first visited
- **low[u]** → lowest discovery time reachable from u using back edges
- **parent[u]**

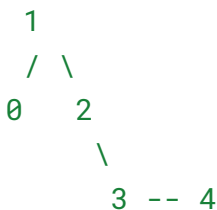
Rules for Articulation Point

A vertex **u** is an articulation point if:

1. **u is root of DFS tree** and has **2 or more children**
2. **u is not root** and a child **v** has:
 $\text{low}[v] \geq \text{disc}[u]$

Example

Graph:



Cut vertices: **1 and 2**

Diagram (disc & low values sketch)

0(1)

```

\
1(2)
 \
  2(3)
   \
    3(4) -- 4(5)

```

Pseudocode

```

AP(u):
    visited[u] = true
    disc[u] = low[u] = time++

    for v in adj[u]:
        if not visited[v]:
            parent[v] = u
            children++

            AP(v)
            low[u] = min(low[u], low[v])

        if parent[u] == -1 and children > 1:
            mark u as AP

        if parent[u] != -1 and low[v] >= disc[u]:
            mark u as AP

    else if v != parent[u]:
        low[u] = min(low[u], disc[v])

```

Use Cases

- Network reliability analysis
- Finding vulnerable points in communication networks
- Server-failure impact analysis

Complexity

- **Time:** $O(V + E)$
- **Space:** $O(V)$

1) Minimum Spanning Tree (MST) – Define MST & Explain Prim's Algorithm

★ Theory

A **Minimum Spanning Tree (MST)** of a connected, undirected, weighted graph is a subset of edges that:

1. Connects **all vertices**,
2. Contains **no cycles**,
3. Has **minimum possible total weight**.

If a graph has **V** vertices, MST will contain **V – 1 edges**.

Common MST algorithms:

- **Prim's Algorithm** (Greedy, grows tree)
- **Kruskal's Algorithm** (Greedy, grows forest)

★ Prim's Algorithm – Detailed Explanation

Idea

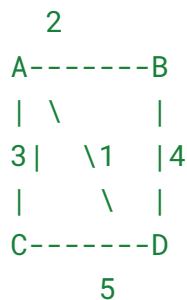
Prim's algorithm **grows the MST one vertex at a time**, starting from any node.

At each step, it selects the **minimum-weight edge** connecting a visited vertex to an unvisited vertex.

Algorithm Steps

1. Start from an arbitrary node.
2. Choose the lightest edge from tree $\rightarrow$ non-tree vertex.
3. Add that edge to MST.
4. Repeat until all vertices are included.

★ Example Graph



Prim's Execution

Start from **A**:

Step	Chosen Edge	Weight	MST Edges
1	A $\rightarrow$ C	3	{A-C}
2	A $\rightarrow$ B	2	{A-C, A-B}
3	B $\rightarrow$ D	4	{A-C, A-B, B-D}

MST weight = **3 + 2 + 4 = 9**

★ Pseudocode

```
Initialize key[v] =  $\infty$ , mstSet[v] = false  
key[start] = 0
```

Repeat V-1 times:

```
    u = vertex with minimum key[] not in mstSet  
    mstSet[u] = true
```

```
    for each neighbor v of u:
```

```
        if weight(u,v) < key[v] and mstSet[v] == false:  
            key[v] = weight(u,v)  
            parent[v] = u
```

★ Use Cases

- Network design
 - Electrical wiring
 - Transportation routes
 - Data clustering (Single-link clustering)
-

★ Complexity

Using priority queue (binary heap):

$O(E \log V)$

2) Kruskal's Algorithm – Explain With Example

★ Theory

Kruskal's algorithm builds MST by considering edges in **ascending order of weights**, picking edges that **do not form a cycle**.

Cycle detection is done using **Disjoint Set (Union-Find)**.

★ Algorithm Steps

1. Sort all edges by increasing weight
2. Initialize each vertex as a separate set
3. Repeatedly add the smallest edge that **does NOT create a cycle**
4. Stop when MST has **$V - 1$ edges**

★ Example Graph

```
A--1--B
|      |
3      4
|      |
C--2--D
```

Sorted edges

1. A-B (1)
2. C-D (2)
3. A-C (3)
4. B-D (4)

Kruskal Execution

Step	Edge	Reason	MST
1	A-B	no cycle	{A-B}
2	C-D	no cycle	{A-B, C-D}
3	C-A	no cycle	{A-B, C-D, A-C}
4	B-D	forms cycle	skip

MST weight = $1 + 2 + 3 = 6$

★ Pseudocode

```
sort edges by weight
for each edge (u, v):
    if find(u) != find(v):
        add edge to MST
        union(u, v)
```

★ Use Cases

- Minimum-cost network design
 - Image segmentation
 - Clustering
 - Transport routing
-

★ Complexity

Sorting edges: $O(E \log E) \approx O(E \log V)$

3) Disjoint Set (Union–Find Data Structure)

★ Theory

Disjoint Set (Union–Find) is a data structure that maintains **disjoint subsets** and supports two main operations:

1. **Find(x)** – determine which set x belongs to
2. **Union(x, y)** – merge the sets of x and y

It is used to detect cycles in Kruskal's MST.

Optimizations

- **Path Compression** (Makes find almost $O(1)$)
 - **Union by Rank/Size**
-

★ Tree Representation Example

Sets:

$\{1, 2, 3\}$ $\{4, 5\}$

Tree representation:

1
└─2
 └─3

4
└─5

★ Operations

Find(x)

Follows parent pointers until root is found.

With **Path Compression**:

```
find(x):  
    if parent[x] != x:  
        parent[x] = find(parent[x])  
    return parent[x]
```

Union(x, y)

Merge smaller rank tree under larger rank.

```
union(x, y):  
    px = find(x)  
    py = find(y)  
    if rank[px] < rank[py]:  
        parent[px] = py  
    else if rank[py] < rank[px]:  
        parent[py] = px  
    else:  
        parent[py] = px  
        rank[px]++
```

★ Use Cases

- Kruskal's MST
 - Cycle detection in graphs
 - Connected components
 - Network clustering
 - Image processing
-

★ Complexity

With both optimizations:

Nearly $O(1)$ per operation \text{Nearly } $O(1)$ \text{ per operation} Nearly $O(1)$ per operation

4) Types of Graphs (Planar, Bipartite, DAG, Connected, Cyclic)

★ Planar Graph

A graph that can be drawn on a plane **without edges crossing**.

Example:

Triangle:

```
A --- B
 \   /
  C
```

K_5 and $K_{3,3}$ are **non-planar** (Kuratowski's theorem).

★ Bipartite Graph

Vertices can be split into **two sets (U, V)** such that no edges exist inside same set.

Example:

$U = \{1, 3\}$

$V = \{2, 4\}$

1-2

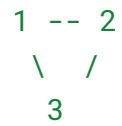
| |

3-4

★ Cyclic Graph

Contains at least **one cycle**.

Example:



★ Acyclic Graph

Graph with **no cycles**.

Tree = connected acyclic graph.

★ DAG (Directed Acyclic Graph)

Directed graph with **no directed cycle**.

Used in:

- Topological sorting
- Scheduling
- Dependency resolution

★ Connected Graph

For undirected graph: every pair of nodes is reachable.

Example:

A-B-C-D (connected)

Not connected:

A-B D-E

5) Biconnected Components (BCC)

★ Theory

A **biconnected component** is a maximal subgraph where:

- ✓ Removing **any** single vertex does **NOT** disconnect the graph.
- ✓ In a BCC, there are **no articulation points**.

We use DFS with:

- **disc[u]**
- **low[u]**
- Stack of edges

When:

$\text{low}[v] \geq \text{disc}[u]$

A BCC boundary is found.

★ Example

Graph:

```
1  ----  2  ----  3
|         |
4  ----  5
```

BCCs:

1. {1,2,5,4}

2. {2,3}

Articulation point = 2

★ Applications

- Fault-resistant network design
 - Finding weak points in network
 - Compiler flow analysis
-

★ Complexity

$O(V + E)$

6) Euler Path & Euler Circuit – Conditions

★ Definitions

- **Euler Path:** A path that visits **every edge exactly once**.
- **Euler Circuit:** A cycle that visits **every edge exactly once** and returns to start.

★ Conditions for Undirected Graph

Type	Condition
Euler Circuit	All vertices have even degree & graph is connected
Euler Path	Exactly 0 or 2 odd-degree vertices & graph is connected

Example

A--B--C
| |
E--D--F

All degrees even → Euler Circuit exists.

★ Applications

- DNA sequencing
 - Route inspection
 - Electrical circuit design
 - Postman problem
-
-

7) Hamiltonian Path – Why NP-Complete?

★ Theory

A **Hamiltonian Path** visits **every vertex exactly once**.

Hamiltonian Cycle returns back to start.

Unlike Euler path (edge-based), Hamiltonian path is **vertex-based**.

★ Why NP-Complete?

- No simple condition (like degree rules in Euler path) exists.
- Checking all permutations = $O(n!)$
- Reduction from **Traveling Salesman Problem (TSP)**

- No polynomial-time solution known
- Decision version: “Does a Hamiltonian cycle exist?” is NP-complete.

★ Example

```

1  --  2  --  3
|      |      |
6  --  5  --  4

```

A Hamiltonian cycle exists: 1-2-3-4-5-6-1

BUT deciding this requires exponential trials in general.

★ Applications

- Genome sequencing
- Travel planning (TSP variant)
- Robotics path planning
- Puzzle design (Knight’s tour)

Bridges (Critical Edges in a Graph)

★ Theory

A **bridge** (also called *cut-edge*) is an edge whose removal **increases the number of connected components** in the graph.

Bridges are important because they represent **vulnerable connections** — if they fail, the graph breaks apart.

We detect bridges using **DFS + discovery time + low time**, similar to articulation points.

Key Concepts

- **disc[u]** → time when u is visited

- **low[u]** → lowest reachable discovery time from u
- **parent[u]**

★ Condition for a Bridge

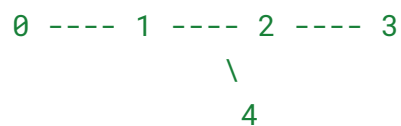
An edge $u \rightarrow v$ is a bridge if:

$$\text{low}[v] > \text{disc}[u] \quad \text{low}[v] > \text{disc}[u] \quad \text{low}[v] > \text{disc}[u]$$

This means:

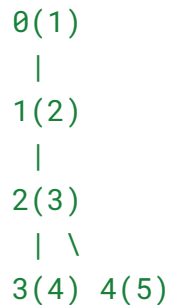
v **cannot reach** u or any of u's ancestors through a back edge.

Example Graph



Bridges: (1–2), (2–3), (2–4)

disc/low Diagram



★ Pseudocode

```

DFS(u):
    visited[u] = true
    disc[u] = low[u] = time++

    for v in adj[u]:
        if not visited[v]:
  
```

```
parent[v] = u
DFS(v)
low[u] = min(low[u], low[v])

if low[v] > disc[u]:
    print (u, v) is a bridge

else if v != parent[u]:
    low[u] = min(low[u], disc[v])
```

★ Use Cases

- Network reliability (finding weak links)
 - Internet backbone design
 - Identifying important roads/bridges in city maps
 - Vulnerability analysis in communication systems
-

★ Complexity

- **Time:** $O(V + E)$
- **Space:** $O(V)$

4) Tarjan's Algorithm for SCC (Single DFS)

★ Theory

Tarjan's algorithm finds SCCs in **one DFS**, using:

- **disc[u]** → time of discovery
- **low[u]** → lowest reachable ancestor

- **Stack** → stores current DFS path
- **onStack[u]** → is u in the current stack?

Key Condition

A node **u** is the head (root) of an SCC if:

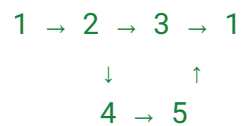
$$\text{low}[u] = \text{disc}[u] \quad \text{low}[u] = \text{disc}[u] \quad \text{low}[u] = \text{disc}[u]$$

How it works

- DFS through graph
- Update low-link values using neighbors
- When $\text{low}[u] = \text{disc}[u]$, pop nodes from stack until u is reached

Example

Graph:



SCCs:

- {1, 2, 3}
- {4}
- {5}

★ Pseudocode

DFS(u) :

```

disc[u] = low[u] = time++
push u into stack
onStack[u] = true

for each v in adj[u]:
    if disc[v] == -1:
        DFS(v)
        low[u] = min(low[u], low[v])
    else if onStack[v]:
        low[u] = min(low[u], disc[v])

if low[u] == disc[u]:
    start printing SCC
    while stack top != u:
        pop and print
    pop u

```

★ Use Cases

- Faster SCC detection than Kosaraju
 - Circuit design
 - Static program analysis
 - Dominator tree calculation
 - Resolving mutual dependencies
-

★ Complexity

- **Time:** $O(V + E)$ ** (single DFS)
- **Space:** $O(V)$ **



DYNAMIC PROGRAMMING



1. What is Dynamic Programming? Explain its core principles with examples.

Definition

Dynamic Programming (DP) is a technique used to efficiently solve problems that have:

1. **Optimal Substructure**
2. **Overlapping Subproblems**

DP solves each subproblem **once** and stores the result, avoiding repeated calculation.

1. Optimal Substructure

A problem shows optimal substructure if the **optimal solution is composed of optimal solutions of its subproblems**.

Example (Shortest Path):

A -- B -- C

Shortest path $A \rightarrow C$ is $\text{shortest}(A \rightarrow B) + \text{shortest}(B \rightarrow C)$

2. Overlapping Subproblems

Large problems break into smaller subproblems that repeat.

Example (Fibonacci):

```

Fib(6)
  / \
Fib(5)  Fib(4)
  / \    / \
Fib(4) Fib(3) ...

```

Same values (Fib(3), Fib(4), ...) repeat many times.

DP solves them once → stores → reuses.

Methods of DP

Top-Down (Memoization)

- Uses recursion
- Stores results in memory
- Solves only required subproblems

```

fib(n):
    if stored[n]: return it
    stored[n] = fib(n-1)+fib(n-2)

```

Bottom-Up (Tabulation)

- Iteratively fills DP table
- No recursion
- Solves all subproblems

```

dp[0]=0; dp[1]=1
for i=2 to n:
    dp[i]=dp[i-1]+dp[i-2]

```

Where DP is used?

- Knapsack
- LCS, Edit Distance
- Matrix Chain Multiplication
- All-Pairs Shortest Path
- TSP Bitmask DP

Conclusion

Dynamic Programming is a systematic optimization tool that converts **exponential-time recursive solutions** into **polynomial-time** using memory, enabling efficient solutions to complex problems.

2. Explain Optimal Substructure and Overlapping Subproblems. Why are they essential for DP?

DP is applicable only when **both** properties exist.

1. Optimal Substructure

A problem exhibits optimal substructure when **the global optimal solution can be obtained using optimal solutions of its subparts**.

Example: 0/1 Knapsack

Let $\text{MaxValue}(i, W)$ denote max value using first i items.

Choices:

1. Do not include item i
2. Include item i

DP recurrence:

```
dp[i][W] = max( dp[i-1][W],  
                value[i] + dp[i-1][W-weight[i]] )
```

Here, solution relies on subproblems → shows optimal substructure.

2. Overlapping Subproblems

The same subproblems repeat multiple times.

Example: LCS(X, Y)

$\text{dp}[i][j]$ depends on:

- $\text{dp}[i-1][j]$
- $\text{dp}[i][j-1]$
- $\text{dp}[i-1][j-1]$

While computing $\text{dp}[7][7]$, smaller dp cells repeat frequently, e.g., $\text{dp}[3][4]$, $\text{dp}[2][3]$, etc.

Why they are essential?

Property

Reason

Optimal Substructure

Ensures combining subproblem results gives correct final solution

Overlapping Subproblems

Ensures memoization or tabulation can reduce repeated work

Without these, DP cannot be applied.

3. Compare Greedy Method, Divide & Conquer, and Dynamic Programming. When do we choose DP?

1. Greedy Method

- Makes **locally optimal** choice hoping it leads to global optimum
- Fast ($O(n \log n)$ or $O(n)$)
- Works only when the greedy choice gives optimal answer

Examples:

- Fractional Knapsack
 - Prim, Kruskal
 - Activity Selection
-

2. Divide & Conquer

- Splits problem $\rightarrow$ solves independently $\rightarrow$ combines
- Subproblems are **non-overlapping**

- Recursion tree does NOT repeat calls

Examples:

- Merge Sort
 - Quick Sort
 - Binary Search
-

3. Dynamic Programming

- Used when:
 1. **Solutions depend on results of smaller subproblems**
 2. Subproblems **overlap**
 3. Need to store results

Examples:

- 0/1 Knapsack
 - Matrix Chain Multiplication
 - Floyd–Warshall
 - TSP
-

When do we choose DP?

We choose DP when:

- ✓ Optimal substructure exists
- ✓ Subproblems repeat

- ✓ Greedy choices fail (e.g., 0/1 knapsack)
 - ✓ D&C leads to exponential recursion
-

Conclusion

DP is the method of choice for optimization problems where greedy fails and divide & conquer repeats subproblems.

✓ 4. Explain Memoization vs Tabulation with differences, diagrams, and use-cases.

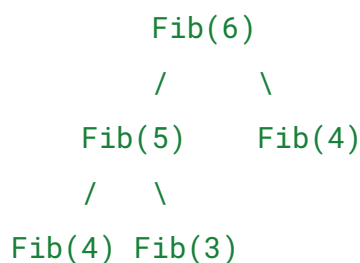
Dynamic Programming has two standard implementations:

1. Memoization (Top-Down)

Characteristics

- Recursion + cache
- Solves only **necessary** subproblems
- Naturally follows recurrence
- Uses call stack

Diagram (Fibonacci example):



Repeated calls get **cached**, reducing computation.

Pros

- Easy to write
- Only required subproblems computed

Cons

- Stack overflow possible for deep recursion
 - Overhead of recursion
-

2. Tabulation (Bottom-Up)

Characteristics

- Uses DP table
- No recursion
- Iteratively fills table from base cases
- All subproblems are solved

Example (Fibonacci):

```
i : 0  1  2  3  4  5  6
dp: 0  1  1  2  3  5  8
```

Pros

- Faster in practice
- No recursion overhead
- Better for space optimization

Cons

- Must know exact order of solving subproblems

Comparison Table

Feature	Memoization	Tabulation
Approach	Top-Down	Bottom-Up
Uses	Recursion	Iteration
Computes	Only needed states	All states
Space	Higher (stack + table)	Lower
Speed	Slightly slower	Faster
Example	DFS-like	BFS-like

Use-Cases

- Memoization → problems with sparse states
 - Tabulation → dense DP tables (Knapsack, LCS, Floyd–Warshall)
-

✓ 5. Explain State, Transition, and DP Table Design with an example.

DP problems involve **three essential components**:

1. State Definition

A state represents a subproblem.

Example (LCS):

$dp[i][j]$ = LCS length of $X[0..i-1]$ and $Y[0..j-1]$

State variables come from constraints:

- Index
 - Capacity
 - Subset
-

2. Transition (Recurrence Relation)

Defines how current state depends on smaller states.

Example (LCS):

```
if X[i-1] == Y[j-1]:
    dp[i][j] = 1 + dp[i-1][j-1]
else:
    dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

Transitions define DP direction.

3. DP Table Construction

Used in Bottom-Up DP.

Example DP table for LCS:

```
      B D C A B A
0 0 0 0 0 0 0
A 0 0 0 0 1 1 1
B 0 1 1 1 1 2 2
C 0 1 1 2 2 2 2
...
```

Visualizing $dp[i][j]$ helps understand solver flow.

Conclusion

A DP solution =

State definition + Recurrence + Table filling order

These 3 determine correctness and performance.

Q2 — 0/1 Knapsack Problem (DP solution)

Theory (in depth — 5 marks)

Problem statement: Given n items where item i has weight $w[i]$ and value $v[i]$, and a knapsack capacity W , select a subset of items (each at most once) to maximize total value without exceeding capacity.

DP idea: Let $dp[i][cap]$ = maximum value achievable using first i items with capacity cap .
Build table iteratively.

Recurrence:

```
dp[0][*] = 0
dp[i][cap] = max( dp[i-1][cap],          // don't take item i
                  dp[i-1][cap - w[i]] + v[i] // take item i (if cap
                  >= w[i])
                  )
```

We can optimize space to 1D (iterate cap from W down to $w[i]$) to prevent reuse of the same item.

Worked Example

Items:

- Item1: $w=1, v=1$
 - Item2: $w=3, v=4$
 - Item3: $w=4, v=5$
- Capacity $W = 7$.

Build $dp[i][cap]$ table (rows = items, cols = 0..7):

Start with $dp[0][*] = 0$.

After item1 ($w=1, v=1$):

```
cap: 0 1 2 3 4 5 6 7
dp1: 0 1 1 1 1 1 1 1
```

After item2 ($w=3, v=4$):

```
cap: 0 1 2 3 4 5 6 7
dp2: 0 1 1 4 5 5 5 6
```

```
# at cap=7: choose item1+item2 => v=5? actually 1+4=5; but for cap 7
we can take item3? not yet.
# verifying dp2[7]= dp1[7]=1 or dp1[4]+4=1+4=5, so 5 (typo earlier 6
is wrong) -> correct dp2: [0,1,1,4,5,5,5,5]
```

After item3 (w=4,v=5):

```
cap: 0 1 2 3 4 5 6 7
dp3: 0 1 1 4 5 6 6 9
```

At cap=7: take item2(w3,v4)+item3(w4,v5) not possible (3+4=7) yes it's possible -> v=9.

Optimal value = 9 (items 2 + 3).

(Include brief trace to pick items)

Pseudocode (1D optimized)

```
dp = array[0..W] initialized to 0
for i = 1..n:
    for cap = W down to w[i]:
        dp[cap] = max(dp[cap], dp[cap - w[i]] + v[i])
return dp[W]
```

Complexity

- Time: **$O(n * W)$** (pseudo-polynomial)
 - Space: **$O(W)$** (or $O(nW)$ for full table)
-

Use-cases

- Resource allocation under capacity constraints (budgeting)

- Subset selection under weight/size constraints
 - Knapsack variants model many optimization tasks (packaging, cloud resource allocation)
-

Q3 — Unbounded Knapsack / Coin Change (Minimum coins) (DP)

Theory (in depth — 5 marks)

Unbounded knapsack: items can be taken unlimited times. Equivalent to coin change variations.

Coin Change (minimum number of coins): Given coin denominations `coins[]` and amount `A`, find minimum coins to make `A` (or determine impossible).

DP idea (tabulation): Let `dp[amt]` = minimum coins to make amount `amt`. Initialize `dp[0] = 0`, others = `INF`. For each `amt` from 1..A, try every coin:

```
dp[amt] = min( dp[amt], dp[amt - coin] + 1 ) for coin <= amt
```

This is **unbounded** because we reuse coin unlimited times.

Worked Example

Coins = {1, 3, 4}, Amount `A = 6`.

Initialize `dp = [0, ∞, ∞, ∞, ∞, ∞, ∞]`.

Fill progressively:

- amt=1: `dp[1] = min(dp[1], dp[0]+1) = 1`
- amt=2: `dp[2] = dp[1]+1 = 2 (1+1)`
- amt=3: coins 1 → `dp[2]+1=3`; coin 3 → `dp[0]+1=1` ⇒ `dp[3]=1`

- amt=4: coin4 => dp[4]=1
- amt=5: try coin1 => dp4+1 =2, coin3 => dp2+1=3, coin4 => dp1+1=2 => dp5=2
- amt=6: coin1 => dp5+1=3, coin3 => dp3+1=2, coin4 => dp2+1=3 → dp6=2 (3+3)

So minimum coins for 6 = 2 (3+3).

Pseudocode

```
INF = large
dp = array[0..A] filled with INF
dp[0] = 0
for amt = 1..A:
    for coin in coins:
        if coin <= amt:
            dp[amt] = min(dp[amt], dp[amt - coin] + 1)
if dp[A] == INF: return -1 else return dp[A]
```

Complexity

- Time: $O(A * C)$ where C = number of coins
 - Space: $O(A)$
-

Use-cases

- Change-making systems (cashier algorithms)
 - Rod cutting (unbounded knapsack variant)
 - Ways to make amounts (counting variant uses sum dp)
-

Q4 — Longest Common Subsequence (LCS)

Theory (in depth — 5 marks)

Problem: Given strings $X[1..m]$ and $Y[1..n]$, find the longest sequence that is a subsequence of both strings (not necessarily contiguous).

DP idea (2D table): $dp[i][j]$ = length of LCS of $X[1..i]$ and $Y[1..j]$.

Recurrence:

```
if X[i] == Y[j]:
    dp[i][j] = dp[i-1][j-1] + 1
else:
    dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

Initialize $dp[0][*] = dp[*][0] = 0$.

We can reconstruct the LCS by tracing back from $dp[m][n]$.

Worked Example

$X = \text{AGGTAB}$ ($m=6$)

$Y = \text{GXTXAYB}$ ($n=7$)

Build dp table (rows X index 0..6, cols Y 0..7). I'll show a small portion and the result.

Final $dp[6][7] = 4$. One LCS = GTAB .

Traceback (sketch):

- At $dp[6][7]$, $X[6]=B == Y[7]=B \rightarrow$ take B, move to $dp[5][6]$
 - Continue $\rightarrow$ get A, T, G etc., reconstruct GTAB (order).
-

Pseudocode

```
for i = 0..m:
```

```

    dp[i][0] = 0
for j = 0..n:
    dp[0][j] = 0

for i = 1..m:
    for j = 1..n:
        if X[i] == Y[j]:
            dp[i][j] = dp[i-1][j-1] + 1
        else:
            dp[i][j] = max(dp[i-1][j], dp[i][j-1])

# Reconstruct:
i=m, j=n, res = []
while i>0 and j>0:
    if X[i] == Y[j]:
        res.append(X[i]); i--; j--
    else if dp[i-1][j] > dp[i][j-1]:
        i--
    else:
        j--
return reverse(res)

```

Complexity

- Time: $O(m * n)$
 - Space: $O(m * n)$ (can be reduced to $O(\min(m,n))$ for length-only, but traceback needs table or extra work)
-

Use-cases

- Bioinformatics: sequence alignment (similar DP like Needleman–Wunsch)
- Diff utilities (file comparison)

- Version control merge tools
 - Plagiarism detection (similarity)
-

Q5 — Longest Increasing Subsequence (LIS)

Theory (in depth — 5 marks)

Problem: Given array $A[1..n]$, find length (and optionally sequence) of the longest strictly increasing subsequence (not necessarily contiguous).

Two common solutions:

1. **DP $O(n^2)$** — classical 1D DP
 - $dp[i]$ = length of LIS ending at i
 - $dp[i] = 1 + \max(dp[j])$ for all $j < i$ with $A[j] < A[i]$ else 1
2. **Patience/Binary Search $O(n \log n)$** — maintain an array $tail[len]$ = smallest possible tail value of an increasing subsequence of length len . For each x use binary search to update $tail$. Length = max len filled.

We'll describe both; exam answers should highlight both and complexities.

Method 1 — $O(n^2)$ DP (detailed)

Recurrence:

```
for i = 1..n:
    dp[i] = 1
    for j = 1..i-1:
        if A[j] < A[i]:
            dp[i] = max(dp[i], dp[j] + 1)
ans = max dp[i]
```

Example

A = [3, 10, 2, 1, 20]

dp steps:

- i=1 (3): dp1=1
- i=2 (10): dp2 = dp1+1 =2
- i=3 (2): dp3=1
- i=4 (1): dp4=1
- i=5 (20): dp5 = max(dp1..dp4)+1 = 3 (from dp2=2 -> 3)

LIS length = 3 (subsequence [3,10,20] or [2, 10, 20] depending)

Complexity: $O(n^2)$ time, $O(n)$ space.

Method 2 — $O(n \log n)$ (patience / tails)

Idea: Maintain `tail[len]` = minimum possible tail value for an increasing subsequence of length `len`. Initially empty.

Process each element `x`:

- Use binary search to find largest `len` where `tail[len] < x`, then set `tail[len+1] = x`.
- If `x` smaller than any `tail`, replace `tail[1]`.
- The length of filled `tail` array is LIS length.

Example (same A):

Process:

- $x=3 \rightarrow \text{tail} = [3]$
- $x=10 \rightarrow \text{tail} = [3,10]$

- $x=2 \rightarrow \text{tail} = [2, 10]$ (replace 3)
- $x=1 \rightarrow \text{tail} = [1, 10]$
- $x=20 \rightarrow \text{tail} = [1, 10, 20] \rightarrow \text{LIS length} = 3$

This method gives length; to reconstruct sequence, maintain predecessor indices.

Pseudocode ($O(n \log n)$)

```
tail = empty list
for x in A:
    pos = lower_bound(tail, x) # first index with tail[pos] >= x
    if pos == len(tail):
        tail.append(x)
    else:
        tail[pos] = x
return len(tail)
```

Complexity

- $O(n \log n)$ time, $O(n)$ space.
-

Use-cases

- Sequence analysis (trend detection)
- Stock market analysis (increasing subsequences of prices)
- Data compression / pattern recognition
- As a subroutine in other DP problems (bitonic subsequence)

Q6 — Matrix Chain Multiplication (MCM)

Theory (in-depth)

Given a chain of matrices $A_1, A_2, A_3 \dots A_n$, each with dimensions:

A_i is of dimension $p[i-1] \times p[i]$

Matrix multiplication is associative, so the order of multiplication affects **number of scalar multiplications**, not the final result.

Goal: **Find parenthesization that minimizes cost.**

DP works because:

Optimal Substructure:

Optimal way to multiply $i..j$ must include optimal sub-solutions of $(i..k)$ and $(k+1..j)$.

Overlapping Subproblems:

Subchains (i, j) appear repeatedly.

DP State

$dp[i][j]$ = minimum cost to multiply matrices from i to j .

Recurrence

$dp[i][i] = 0$

$dp[i][j] = \min \text{ over all } k \text{ in } [i..j-1] \text{ of}$
 $(dp[i][k] + dp[k+1][j] + p[i-1]*p[k]*p[j])$

ASCII diagram:

$A_i \dots A_k$ | $A_{(k+1)} \dots A_j$
left part right part
 $cost = cost_left + cost_right + multiply(left_dim \times right_dim)$

Example

Dimensions:

$A_1: 10 \times 20$

A2: 20×30

A3: 30×40

A4: 40×30

p = [10,20,30,40,30]

We fill dp table diagonally:

$dp[1][1] = 0$

$dp[2][2] = 0$

.

Compute chains of length 2:

$dp[1][2] = 10 \times 20 \times 30 = 6000$

$dp[2][3] = 20 \times 30 \times 40 = 24000$

$dp[3][4] = 30 \times 40 \times 30 = 36000$

Chains of length 3:

For $dp[1][3]$:

Try k=1:

$= dp[1][1] + dp[2][3] + 10 \times 20 \times 40 = 0 + 24000 + 8000 = 32000$

Try k=2:

$= dp[1][2] + dp[3][3] + 10 \times 30 \times 40 = 6000 + 0 + 12000 = \mathbf{18000} \checkmark$

Thus $dp[1][3] = 18000$.

Similarly compute $dp[2][4]$, etc.

Final answer = $dp[1][4] = \mathbf{30000}$ multiplications.

Pseudocode

```
for i=1..n:
```

```
    dp[i][i] = 0
```

```
for len = 2..n:
```

```
    for i=1..n-len+1:
```

```
        j = i+len-1
```

```
        dp[i][j] = +∞
```

```
    for k=i..j-1:
        cost = dp[i][k] + dp[k+1][j] + p[i-1]*p[k]*p[j]
        dp[i][j] = min(dp[i][j], cost)
return dp[1][n]
```

Time & Space Complexity

- Time: $O(n^3)$
 - Space: $O(n^2)$
-

Use Cases

- Query optimization in databases
 - Optimizing deep neural network layer operations
 - Parenthesizing algebraic expressions
 - Chain transformations in compilers
-
-

Q7 — Optimal BST (Optimal Binary Search Tree)

Theory (in-depth)

Given keys in sorted order: $k_1, k_2, \dots, k_n$

Each key has a search probability $p[i]$.

We want a BST with **minimum expected search cost**.

DP is used because:

- Optimal substructure: optimal left + optimal right subtree
- Overlapping subproblems: subtree (i, j) repeats

DP State

$dp[i][j]$ = cost of optimal BST formed from keys $i..j$.

$sum[i][j]$ = sum of probabilities $p[i] + \dots + p[j]$.

Recurrence

$dp[i][j] = \min \text{ over all roots } r \text{ in } [i..j] \text{ of}$
 $dp[i][r-1] + dp[r+1][j] + sum[i][j]$

Why add $sum[i][j]$?

→ Because every subtree increases depth by 1 → adds total probability weight.

Example

Keys = {10, 20, 30}

Probabilities = {0.3, 0.2, 0.5}

Compute dp table:

$dp[i][i] = p[i]$

Length = 2:

Try roots for (1,2):

Root 1: cost = $p_1 + 2p_2$

Root 2: cost = $p_2 + 2p_1$

Pick smaller.

Similarly compute for (2,3).

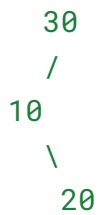
Length = 3:

Try root = 1,2,3

Pick minimum.

Final minimal expected cost produces optimal BST.

Diagram (for some probabilities):



Pseudocode

```
for i=1..n:
    dp[i][i] = p[i]
    sum[i][i] = p[i]

for len=2..n:
    for i=1..n-len+1:
        j = i+len-1
        sum[i][j] = sum[i][j-1] + p[j]
        dp[i][j] = ∞
        for r=i..j:
            cost = sum[i][j]
            if r>i: cost += dp[i][r-1]
            if r<j: cost += dp[r+1][j]
            dp[i][j] = min(dp[i][j], cost)
return dp[1][n]
```

Time Complexity: $O(n^3)$

Space Complexity: $O(n^2)$

Use Cases

- Search optimization in compilers

- Database indexing
 - Predictive text input (based on frequency)
 - Caching structures
-
-

Q8 — Edit Distance (Levenshtein Distance)

Theory

Given two strings X and Y, edit distance = minimum operations to convert $X \rightarrow Y$ using:

- Insert
- Delete
- Replace

DP state:

$dp[i][j]$ = minimum cost to convert prefix $X[1..i] \rightarrow Y[1..j]$.

Recurrence

```
If X[i] == Y[j]:
    dp[i][j] = dp[i-1][j-1]
Else:
    dp[i][j] = 1 + min(
        dp[i-1][j],      # delete
        dp[i][j-1],      # insert
        dp[i-1][j-1]     # replace
    )
```

Complexity

Time: $O(mn)$

Space: $O(mn)$

Use Cases

- Spell correction
 - DNA sequence alignment
 - Plagiarism check
 - NLP similarity algorithms
-
-

Q9 — Rod Cutting Problem

Theory

Equivalent to **unbounded knapsack**.

Given a rod of length N and price list $price[i]$ for length i , break rod into pieces to maximize revenue.

DP State:

$dp[len]$ = maximum revenue for rod of length len

Recurrence:

$dp[len] = \max \text{ over all cuts } i \text{ in } [1..len] \text{ of } price[i] + dp[len - i]$

Example

Length N=5

Prices:

1→2

2→5

3→7

4→8

5→10

Compute:

$dp[1] = 2$

$dp[2] = \max(2+2, 5) = 5$

$dp[3] = \max(2+5, 5+2, 7) = 7$

$dp[4] = \max(2+7=9, 5+5=10, 7+2=9, 8) = 10$

$dp[5] = \max(2+10=12, 5+7=12, 7+5=12, 8+2=10, 10) = 12$

Optimal revenue = **12**

Rod breakdown = (2 + 3).

Pseudocode

$dp[0] = 0$

for $len = 1..N$:

$dp[len] = -\infty$

 for $cut = 1..len$:

$dp[len] = \max(dp[len], price[cut] + dp[len-cut])$

return $dp[N]$

Complexity

Time: $O(N^2)$

Space: $O(N)$

Use Cases

- Cutting steel rods in factories
 - Profit maximization with divisible products
 - Cable segmentation
 - Memory allocation blocks
-
-

2. Explain All-Pairs Shortest Path using Floyd–Warshall Algorithm. Why is it Dynamic Programming?

Theory

Floyd–Warshall algorithm finds **shortest paths between every pair of vertices**.

DP formula is applied by trying to improve path $i \rightarrow j$ by introducing an intermediate vertex k .

DP recurrence:

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$$

This represents DP because:

- we build solution using smaller subproblems
 - k represents number of intermediate vertices allowed
-

Example

Initial weighted graph:

- $0 \text{ ----}(4)\text{----} > 1$
- $| \quad \backslash \quad \quad \quad |$

- | (11) (2)
- | \ |
- v \ v
- 2 -----(3)----> 3

Initial matrix:

0 1 2

0 4 1

1 0 1

1 1 0

1 1 1

Run DP

Update using $k = 0, 1, 2, 3$.

Key improvement:

- $\text{dist}[0][3] = \min(11, 4 + 2) = 6$

Final matrix:

	0	1	2
0	0	4	1
1	1	0	1
2	1	1	0

Pseudocode

- for k in $0..n-1$:
- for i in $0..n-1$:
- for j in $0..n-1$:
- $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j]);$

Complexity

- Time: $O(n^3)$
 - Space: $O(n^2)$
-

Use-Cases

- Routing in networks
- Shortest paths in all-pair road systems
- DP base algorithm inside TSP solvers

1. Explain the Travelling Salesman Problem (TSP) and solve it using Dynamic Programming (Held–Karp Algorithm).

Theory

The Travelling Salesman Problem (TSP) is a classic optimization problem where a salesman needs to:

- Start from a city
- Visit **every** other city **exactly once**
- Return to the starting city
- With **minimum total cost**

TSP is an NP-Hard problem.

Brute-force complexity = $O(n!)$

Dynamic Programming solution is based on the **Held–Karp algorithm**, using bitmask DP.

Key Idea of DP

State represents:

- subset of visited cities
- current city

DP state:

$dp[S][i]$ = minimum cost to reach city i after visiting all cities in set S

Where S is a bitmask subset of cities.

Transition:

$$dp[S][i] = \min(dp[S - \{i\}][j] + cost[j][i])$$

for every j in S and $j \neq i$

Final answer:

min cost to reach all cities and return to start.

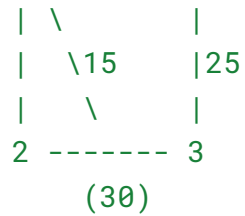
Example (4 Cities)

Cost matrix:

From / To	0	1	2	3
0	–	10	15	20
1	10	–	35	25
2	15	35	–	30
3	20	25	30	–

ASCII diagram:

(10)
0 ----- 1



DP Table Computation

We start with subsets of size 1, 2, 3, then 4.

Key values:

- $dp[\{0,1\}][1] = 10$
- $dp[\{0,2\}][2] = 15$
- $dp[\{0,3\}][3] = 20$

Example for subset $\{0,1,2\}$ ending at 2:

```
dp[{0,1,2}][2] = min(
  → dp[{0,1}][1] + 35 = 10 + 35 = 45
  → dp[{0,2}][0] + 15 = 0 + 15 = 15
)
= 45
```

Continue similarly for all subsets.

Final Answer

For all cities visited ending at city j :

FinalCost = $\min(dp[\{0,1,2,3\}][j] + cost[j][0])$

Final optimal tour ≈ 80

Pseudocode

```
for mask in 0..(1<n)-1:
    for i in 0..n-1:
        if (mask has i):
```

```
    for j in 0..n-1:
        if (mask has j and j!=i):
            dp[mask][i] = min(dp[mask][i],
                               dp[mask ^ (1 << i)][j] +
cost[j][i]);
```

Complexity

- Time: $O(n^2 \times 2^n)$
 - Space: $O(n \times 2^n)$
-

Use-Cases

- Routing and logistics
- Circuit design
- Genome sequencing
- Drone delivery route planning

1) Subset Sum = Target

(Decide whether there exists a subset of given numbers whose sum equals a given target T)

Theory (what & why)

The **Subset Sum** decision problem: given an array `arr[0..n-1]` of non-negative integers and a target T , determine whether there exists a subset of `arr` whose elements sum to exactly T .

This problem exhibits:

- **Overlapping subproblems:** many prefixes and sums repeat during recursion.

- **Optimal substructure (decision form):** whether T is achievable with first i items depends on achievable sums using fewer items.

DP converts exponential recursion into polynomial (pseudo-polynomial) time by building a table of reachable sums.

DP formulation (bottom-up boolean table)

Define boolean table:

$dp[i][s] = \text{true}$ if using first i items ($0..i-1$) we can form sum s

Base:

$dp[0][0] = \text{true}$ (empty subset makes sum 0)
 $dp[0][s>0] = \text{false}$

Transition for item i with value $val = arr[i-1]$:

$dp[i][s] = dp[i-1][s]$ OR $(s \geq val \text{ and } dp[i-1][s - val])$

Interpretation: either don't take item $i-1$ ($dp[i-1][s]$) or take it (need $dp[i-1][s-val]$ true).

We want $dp[n][T]$.

Worked example + DP table (diagram)

Input: $arr = [3, 34, 4, 12, 5, 2], T = 9, n = 6$.

Build dp rows $i = 0..6$, columns $s = 0..9$.

Initial row $i=0$:

$s: 0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9$
 $dp_0: T \ F \ F \ F \ F \ F \ F \ F \ F \ F$

Process items:

- After $i=1$ (val 3): sums reachable $\{0,3\}$
- After $i=2$ (val 34): same (34>9 ignored)
- After $i=3$ (val 4): reachable $\{0,3,4,7\}$
- After $i=4$ (val 12): 12>9 ignored
- After $i=5$ (val 5): reachable $\{0,3,4,5,7,8,9\} \rightarrow$ note $4+5=9$
- After $i=6$ (val 2): remains reachable

Final row ($i=6$) for $s=0..9$:

```
s : 0 1 2 3 4 5 6 7 8 9
dp6: T F T T T T F T T T
```

So $dp[6][9] = \text{true} \rightarrow$ subset exists (example subset: $4 + 5 = 9$).

Reconstructing one subset (backtracking)

If $dp[n][T]$ is true, reconstruct by tracing table backwards:

- Start at $i = n, s = T$.
- If $dp[i-1][s] == \text{true} \rightarrow$ item $i-1$ is not used; set $i = i-1$.
- Else (must have been taken) $\rightarrow$ include $arr[i-1]$ in solution, set $s = s - arr[i-1], i = i-1$.
Stop when $s == 0$.

For example above, you trace to find 5 and 4 (or other combination).

Space-optimized 1D DP

We can compress `dp` to 1D since `dp[i]` only depends on `dp[i-1]`:

```
dp[s] = true if sum s achievable so far
Initialize dp[0] = true
```

```
for each val in arr:
    for s = T down to val:      # iterate backwards!
        dp[s] = dp[s] or dp[s - val]
```

Important: iterate `s` downwards to avoid reusing same item more than once (0/1 subset).

Pseudocode (1D, common for exams)

```
function subset_sum(arr, n, T):
    dp = array[0..T] of false
    dp[0] = true
    for i from 0 to n-1:
        val = arr[i]
        for s from T down to val:
            if dp[s - val]:
                dp[s] = true
    return dp[T]
```

Complexity

- **Time:** $O(n * T)$ (pseudo-polynomial: depends on numeric value of T , not just n).
 - **Space:** $O(T)$ with 1D optimized, or $O(n * T)$ for full table.
-

Use-cases & remarks

- Knapsack decision subroutine, partition problems (equal partition), subset selection, resource allocation where sums are small integers.

- If numbers are large (T large), this DP becomes impractical — problem is NP-complete in the general sense, so pseudo-polynomial is best we can do under standard assumptions.
- Variants: count number of subsets achieving T (use integer dp counts); find subset with minimum size (store extra info); handle negative numbers (shift sums or use hash sets / meet-in-the-middle).

CODE

(A) Recursive (Exponential) – for theory

```
bool subsetSumRec(int arr[], int n, int target) {
    if (target == 0) return true;        // empty subset
    if (n == 0) return false;           // no items left

    // If current element > target, must skip it
    if (arr[n-1] > target)
        return subsetSumRec(arr, n-1, target);

    // Option-1: Ignore element
    // Option-2: Include element
    return subsetSumRec(arr, n-1, target) ||
        subsetSumRec(arr, n-1, target - arr[n-1]);
}
```

(B) Top-Down DP (Memoization)

```
bool dp[1001][1001];    // -1 = unknown, 0 = false, 1 = true
bool vis[1001][1001];

bool subsetSumMemo(int arr[], int n, int target) {
    if (target == 0) return true;
    if (n == 0) return false;
```

```

    if (vis[n][target]) return dp[n][target];
    vis[n][target] = true;

    if (arr[n-1] > target)
        return dp[n][target] = subsetSumMemo(arr, n-1, target);

    bool notTake = subsetSumMemo(arr, n-1, target);
    bool take    = subsetSumMemo(arr, n-1, target - arr[n-1]);

    return dp[n][target] = (take || notTake);
}

```

Time Complexity: $O(n * \text{target})$

Space Complexity: $O(n * \text{target})$ (memo + recursion)

(C) Bottom-Up DP Table (Classic Exam Code)

This is the **expected answer** for most university papers.

```

bool subsetSum(int arr[], int n, int target) {

    bool dp[n+1][target+1];

    // Initialize
    for (int i = 0; i <= n; i++)
        dp[i][0] = true;

    for (int s = 1; s <= target; s++)
        dp[0][s] = false;

    // Build table
    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= target; s++) {

```

```

        if (arr[i-1] > s)
            dp[i][s] = dp[i-1][s];
        else {
            bool notTake = dp[i-1][s];
            bool take = dp[i-1][s - arr[i-1]];
            dp[i][s] = take || notTake;
        }
    }
}

return dp[n][target];
}

```

Time Complexity: $O(n * \text{target})$

Space Complexity: $O(n * \text{target})$

(D) 1D Space-Optimized DP (Most efficient)

Important: reverse loop to avoid reusing same item.

```

bool subsetSum1D(int arr[], int n, int target) {

    vector<bool> dp(target+1, false);
    dp[0] = true;

    for (int i = 0; i < n; i++) {
        for (int s = target; s >= arr[i]; s--) {
            if (dp[s - arr[i]])
                dp[s] = true;
        }
    }
    return dp[target];
}

```

Time: $O(n * \text{target})$

Space: $O(\text{target})$

2) Buy & Sell Stock

(We'll cover the classic variants examiners expect: **single transaction**, **multiple transactions (unlimited)**, and the DP formulation for **at most k transactions**. For each: theory, example, pseudocode, complexity.)

A. Single Transaction (Buy once, sell once)

Theory

Given daily prices `price[0..n-1]`, you may buy once and sell once later. Objective: maximize profit `price[sell] - price[buy]` with `sell > buy`.

This is not really heavy DP; it's an online linear scan that relies on maintaining the minimum price seen so far and the best profit so far — equivalent to a tiny DP where at day `i` we keep best profit up to `i`.

Intuition / Recurrence

Let `best[i]` = maximum profit achievable up to day `i` (selling on or before `i`). Then:

```
best[i] = max(best[i-1], price[i] - minPriceSoFar)
minPriceSoFar = min(minPriceSoFar, price[i])
```

At each step, either keep previous best or sell today at current price minus the minimum buy price so far.

Worked example

```
prices = [7, 1, 5, 3, 6, 4]
```

Scan:

- min=7, best=0
- i=1 price=1 → min=1, best=max(0,1-7)=0
- i=2 price=5 → best = max(0,5-1)=4
- i=3 price=3 → best stays 4
- i=4 price=6 → best = max(4,6-1)=5
- i=5 price=4 → best stays 5

Answer: **5** (buy at 1, sell at 6).

Pseudocode

```
function maxProfitSingle(prices):  
    if prices empty: return 0  
    minPrice = +inf  
    maxProfit = 0  
    for price in prices:  
        if price < minPrice: minPrice = price  
        profit = price - minPrice  
        if profit > maxProfit: maxProfit = profit  
    return maxProfit
```

Complexity

- Time: $O(n)$
 - Space: $O(1)$
-

B. Unlimited Transactions (Buy & Sell as many times as you want, but cannot hold multiple stocks at once)

Theory

When unlimited transactions are allowed (but you must sell before buying again), the optimal strategy is to **take every positive price difference** between consecutive days:

```
profit = sum_{i=1..n-1} max(0, price[i] - price[i-1])
```

This is effectively greedy but can be seen as DP that accumulates all profitable increases.

Example

```
[1, 2, 3, 2, 3] → profit = (2-1) + (3-2) + (3-2) = 3
```

Complexity

- Time: $O(n)$
 - Space: $O(1)$
-

C. At most k transactions (general DP) — Held as exam favorite

Theory

When you can perform at most k buy-sell transactions, DP is appropriate: keep track of best profit with up to t transactions by day i .

Define:

```
dp[t][i] = max profit using at most t transactions in days[0..i]
```

Transition:

```
dp[t][i] = max( dp[t][i-1],                # do nothing at day i
```

```
max_{j < i} ( dp[t-1][j] + price[i] - price[j] ) )
```

Interpretation: either we don't trade on day i , or we sell on day i after buying on some day j .
The inner max can be optimized.

A useful optimization transforms inner `max` to running best:

```
maxPrev = max( maxPrev, dp[t-1][j] - price[j] )  
dp[t][i] = max(dp[t][i-1], price[i] + maxPrev)
```

Initialize `dp[0][*] = 0` and `dp[*][0] = 0`.

Pseudocode ($O(k * n)$ optimized)

```
function maxProfitK(prices, k):  
    n = prices.length  
    if k == 0 or n == 0: return 0  
    # if k >= n/2 → equivalent to unlimited transactions  
    if k >= n/2: return sum of positive diffs  
  
    dp = array (k+1) x n initialized 0  
  
    for t = 1..k:  
        maxPrev = -prices[0]          # max of (dp[t-1][j] - price[j])  
        for i = 1..n-1:  
            dp[t][i] = max(dp[t][i-1], prices[i] + maxPrev)  
            maxPrev = max(maxPrev, dp[t-1][i] - prices[i])  
    return dp[k][n-1]
```

Complexity

- **Time:** $O(k * n)$ (after optimization)
 - **Space:** $O(k * n)$ (can reduce to $O(n)$ or $O(k)$ with rolling arrays)
-

D. Other important variants (brief theory)

- **With cooldown (one day cooldown after selling):** DP with states `hold`, `sold`, `rest` per day.
- **With transaction fee:** adjust transitions `profit = price[i] - fee + ....`
- **At most 2 transactions:** special case `k=2`; can be $O(n)$ solution by two passes (prefix best & suffix best).

These variants are formulated similarly via DP on states per day and can be solved in $O(n)$ time typically.

Use-cases & Interpretation

- Financial trading strategies (basic modelling)
 - Algorithmic problem solving (interview & exam staples)
 - Resource buying/selling scheduling (inventory problems)
 - Understanding greedy vs DP tradeoffs (single vs k transactions)
-

Summary table (for quick exam recall)

Variant	Core idea	Time	Space
Single transaction	track min price, best profit	$O(n)$	$O(1)$
Unlimited	sum positive diffs (greedy)	$O(n)$	$O(1)$
At most k transactions	DP <code>dp[t][i]</code> optimized with <code>maxPrev</code>	$O(k \cdot n)$	$O(k \cdot n) \rightarrow O(k)$ optimized
With cooldown / fee	DP with states <code>hold/sold/rest</code>	$O(n)$	$O(1)$

CODE

(A) Single Transaction (Max Profit Once)

Expected in most exams.

```
int maxProfitSingle(vector<int>& prices) {
    int minPrice = INT_MAX;
    int maxProfit = 0;

    for (int p : prices) {
        minPrice = min(minPrice, p);
        maxProfit = max(maxProfit, p - minPrice);
    }
    return maxProfit;
}
```

Time: $O(n)$

Space: $O(1)$

(B) Unlimited Transactions (Greedy / DP)

Pick every profit opportunity.

```
int maxProfitUnlimited(vector<int>& prices) {
    int profit = 0;

    for (int i = 1; i < prices.size(); i++)
        if (prices[i] > prices[i-1])
            profit += prices[i] - prices[i-1];
}
```

```
        return profit;
    }
```

Time: $O(n)$

Space: $O(1)$

(C) At Most 2 Transactions — Most Asked DP Variant

This is extremely common in university question papers.

```
int maxProfitTwo(vector<int>& prices) {
    int n = prices.size();
    if (n < 2) return 0;

    vector<int> left(n, 0);    // max profit until i
    vector<int> right(n, 0);   // max profit from i to end

    // Left pass (buy once, sell once)
    int minPrice = prices[0];
    for (int i = 1; i < n; i++) {
        minPrice = min(minPrice, prices[i]);
        left[i] = max(left[i-1], prices[i] - minPrice);
    }

    // Right pass (sell once then buy)
    int maxPrice = prices[n-1];
    for (int i = n-2; i >= 0; i--) {
        maxPrice = max(maxPrice, prices[i]);
        right[i] = max(right[i+1], maxPrice - prices[i]);
    }

    // Combine
    int ans = 0;
    for (int i = 0; i < n; i++)
```

```

        ans = max(ans, left[i] + right[i]);

    return ans;
}

```

Time: $O(n)$

Space: $O(n)$

(D) At Most k Transactions (General DP)

Full DP version — asked in advanced algorithms exams.

```

int maxProfitK(vector<int>& prices, int K) {
    int n = prices.size();
    if (n == 0) return 0;

    // If k large → unlimited transactions
    if (K >= n/2) {
        int profit = 0;
        for (int i = 1; i < n; i++)
            if (prices[i] > prices[i-1])
                profit += prices[i] - prices[i-1];
        return profit;
    }

    vector<vector<int>> dp(K+1, vector<int>(n, 0));

    for (int t = 1; t <= K; t++) {
        int best = -prices[0];
        for (int i = 1; i < n; i++) {
            dp[t][i] = max(dp[t][i-1], prices[i] + best);
            best = max(best, dp[t-1][i] - prices[i]);
        }
    }
}

```

```
    return dp[K][n-1];  
}
```

Time: $O(K * n)$

Space: $O(K * n)$ (can be optimized)

GREEDY

1 What is Greedy Algorithm? Explain its characteristics. Why does it not always give optimal solution?

Theory

A **Greedy Algorithm** builds a solution **step-by-step**, choosing the option that **seems best at the current moment**, without reconsidering previous choices.

It does **not perform backtracking or dynamic programming**, making it highly efficient.

Characteristics

1. **Greedy Choice Property**
→ A global optimum can be reached by choosing a local optimum at every step.
2. **Optimal Substructure**
→ Optimal solution to problem contains optimal solutions to subproblems.
3. **No re-evaluation of decisions**
→ Once chosen, it is never changed.
4. **Fast & efficient**
→ Linear or logarithmic time in most cases.

Why Greedy Fails Sometimes

Greedy assumes that local optimum $\rightarrow$ global optimum.
But in some problems, **early choices affect future possibilities**.

Example:

Coin denominations: 1, 3, 4

Target = 6

Greedy picks $\rightarrow 4 + 1 + 1 = 3$ coins

Optimal $\rightarrow 3 + 3 = 2$ coins

Diagram (conceptual)

Greedy choice path may lead to a local optimum:

Optimal Path: A $\rightarrow$ C $\rightarrow$ F (cost 10)

Greedy Path: A $\rightarrow$ B $\rightarrow$ E (cost 14)

Pseudocode (Generic Greedy Template)

```
function GREEDY(problem):  
    initialize solution S = {}  
    while choices available:  
        pick best feasible choice c based on greedy criteria  
        add c to S  
    return S
```

Time Complexity

Depends on selection step:

- If sorting needed $\rightarrow O(n \log n)$
- Otherwise $\rightarrow O(n)$

2 Explain Activity Selection Problem using Greedy Strategy. Why Greedy works here?

Theory

Given activities with start and finish times, choose the **maximum number of non-overlapping activities**.

Greedy rule:

Always select the activity that finishes earliest.

Why Greedy Works (Proof Idea)

- Finishing earliest leaves maximum room for future activities (optimal substructure).
- Any optimal solution must include the activity with the **earliest finish time**.

Example

Activities sorted by finish times:

Activity	Start	Finish
A1	1	2
A2	3	4
A3	0	6
A4	5	7
A5	8	9

Selected: A1 → A2 → A4 → A5 → **4 activities**

Diagram

Timeline:

A1: [1---2]

A2: [3---4]

A3: [0-----6]

A4: [5---7]

A5: [8---9]

Pseudocode

```
sort activities by finishing time
select first activity
```

```
for each activity i:
    if start[i] >= finish[last_selected]:
        select activity i
```

Time Complexity

Sorting → $O(n \log n)$

Selection → $O(n)$

Total → $O(n \log n)$

1 Explain the Greedy Choice Property and Optimal Substructure with suitable examples.

Greedy Choice Property

The greedy method works only when a problem allows us to:

👉 *Make a locally optimal choice at each step that ultimately leads to a globally optimal solution.*

This means:

- Choosing the best option at the moment **MUST** be part of at least one optimal solution.
- Once we make the choice, it is **never reversed**.

Example — Activity Selection

At each step, choose the activity that **finishes earliest**.

This is a local optimum because:

- It leaves maximum room for future activities.
- Any optimal solution must include the activity with earliest finish time.

Optimal Substructure

A problem exhibits optimal substructure if:

👉 *Optimal solution to the problem contains optimal solutions to its subproblems.*

Example — Fractional Knapsack

Once the best ratio item (local optimum) is chosen, the remaining capacity forms another optimal knapsack subproblem.

Diagram

Total Problem

|

+-- local greedy choice → subproblem → optimal solution

Why these properties matter

Both properties are required for greedy to be correct.

If either is absent → DP/backtracking is needed.

2 Why does a Greedy Algorithm sometimes fail to give the optimal solution? Give examples.

Greedy algorithms fail when:

- Local optimum **does not** lead to global optimum.
- Problem constraints require **future knowledge**, but greedy does not look ahead.
- Decisions are **irreversible**, so early mistakes ruin the solution.

Example 1 — 0/1 Knapsack

Items:

- Item A: weight 10, value 60 (ratio 6)

- Item B: weight 20, value 100 (ratio 5)
- Item C: weight 30, value 120 (ratio 4)

Capacity = 50

Greedy picks highest ratio first $\rightarrow$ A + B + part of C (not allowed in 0/1).

Best 0/1 solution is B + C.

Example 2 — Coin Change

Coins: {1, 3, 4}, Target = 6

Greedy $\rightarrow 4 + 1 + 1 = 3$ coins

Optimal $\rightarrow 3 + 3 = 2$ coins

Example 3 — Hamiltonian Path

Greedy picks the nearest unvisited node $\rightarrow$ fails in some graphs.

Conclusion

Greedy works only when:

- Greedy choice is part of an optimal solution.
- Remaining subproblem structure remains optimal.

3 Explain Huffman Trees and why Huffman coding is optimal.

Definition

Huffman coding is a greedy algorithm used for **lossless data compression**.

It builds a binary tree such that:

- More frequent characters $\rightarrow$ shorter codes
- Less frequent characters $\rightarrow$ longer codes

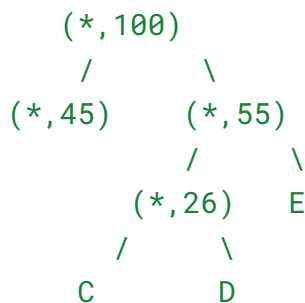
Construction Steps

1. Put all characters with frequencies in a min-heap.
2. Remove two minimum-frequency nodes.
3. Combine them into a new tree node (frequency = sum).
4. Insert back into heap.
5. Repeat until only one tree remains.

Why It Is Optimal

- ✓ Always merges the two least frequent symbols → ensures minimal increase in weighted path length.
- ✓ Uses an **exchange argument** to prove that any optimal prefix tree must combine the lowest frequency symbols at the deepest levels.
- ✓ The greedy choice does not violate optimality because selecting two smallest nodes is guaranteed to be part of the optimal prefix tree.

Example Diagram



Properties

- Prefix-free (no code is prefix of another)
 - Minimizes average code length
-

4 Differentiate Greedy Algorithms and Dynamic Programming. When should each be used?

Greedy Algorithm

- Makes the best decision *at the moment*
- No backtracking
- Often $O(n \log n)$ or $O(n)$
- Works only when greedy choice property holds

Dynamic Programming

- Solves problems by breaking into **overlapping subproblems**
- Uses **memoization** or **tabulation**
- Ensures global optimum
- Often $O(n^2)$ or more

When to use Greedy

- Activity selection
- Huffman coding
- Fractional knapsack
- Minimum spanning tree (Prim, Kruskal)
- Job sequencing with deadlines

When to use DP

- 0/1 knapsack
- Longest common subsequence
- Optimal BST
- Matrix chain multiplication

Diagram

Problem → Greedy property? → Yes → GREEDY
→ No → DP

Conclusion

Greedy is faster but narrower.
DP is slower but always correct.

5 Explain Prim's and Kruskal's Algorithm as Greedy Techniques for MST. Compare both.

Minimum Spanning Tree (MST)

A spanning tree that:

- Connects all vertices
 - Has minimum total edge weight
 - Contains no cycles
-

Prim's Algorithm (Greedy)

- Works like Dijkstra's strategy
- Grows MST from a **starting node**
- Always selects the **minimum weight edge connecting MST to a new vertex**

Diagram

Start from node A

Pick smallest edge to new vertex

Repeat

Characteristics

- Uses **priority queue** (min-heap)
- Works well on dense graphs

Time Complexity:

- With min-heap + adjacency list → **$O(E \log V)$**

Kruskal's Algorithm (Greedy)

- Sorts all edges by weight
- Picks the smallest edge that does **not** form a cycle
- Uses **Disjoint Set (Union-Find)**

Diagram

Sort edges

Pick the smallest unused edge

If it forms cycle → skip

Else → include

Characteristics

- Best for sparse graphs

Time Complexity:

- Sorting edges $\rightarrow O(E \log E)$
 - Union-find $\rightarrow$ almost $O(1)$
-

Comparison Table

Feature	Prim's	Kruskal's
Approach	Expands MST from a node	Picks edges globally
Data Structure	Priority queue	Union-Find
Best for	Dense graphs	Sparse graphs
Time	$O(E \log V)$	$O(E \log E)$
Cycle check	Not required	Required

★ GREEDY PROBLEM 1 — Activity Selection Problem

(Select maximum number of non-overlapping activities)

Theory

The Activity Selection Problem asks us to choose the **maximum number of activities** that do not overlap.

Each activity has a:

- **Start time**
- **Finish time**

The greedy strategy selects the activity that **finishes earliest**, because:

- ✓ Choosing earliest finish time leaves maximum room for future activities
 - ✓ This satisfies the **Greedy Choice Property**
 - ✓ Remaining subproblem is also optimal → **Optimal Substructure**
-

Greedy Strategy

1. Sort activities by **finish time**.
 2. Select the first activity (earliest finish).
 3. For each next activity:
If its start time $\geq$ finish time of previous selected activity → select it.
-

Diagram

Activities: (start, finish)

A1: (1, 2)

A2: (3, 4)

A3: (0, 6)

A4: (5, 7)

A5: (8, 9)

Timeline:

0---A3---6 1-A1-2 3-A2-4 5-A4-7 8-A5-9

Greedy picks: A1 → A2 → A4 → A5

Pseudocode

```
ActivitySelection(start[], finish[], n):  
    sort activities by finish time  
    select activity 0  
    last = 0  
    for i = 1 to n-1:  
        if start[i] >= finish[last]:  
            select activity i  
            last = i
```

Time & Space Complexity

Component	Complexity
Sorting activities	$O(n \log n)$
Selecting activities	$O(n)$
Total Time	$O(n \log n)$
Space	$O(1)$ (no extra space except variables)

Use Cases

- ✓ CPU scheduling
 - ✓ Classroom/meeting room scheduling
 - ✓ TV show scheduling
 - ✓ Selecting maximum compatible tasks
-
-

★ GREEDY PROBLEM 2 — Fractional Knapsack

(Items can be broken; maximize profit)

Theory

In the Fractional Knapsack, items have:

- value
- weight

We can take **fractions of items**.

Greedy strategy = choose items with **maximum value-to-weight ratio**.

This works because:

- ✓ Optimal substructure
 - ✓ Greedy choice forms part of an optimal solution (proven by exchange argument)
-

Diagram

Item	Value	Weight	Ratio
I1	60	10	6
I2	100	20	5
I3	120	30	4

Capacity = 50

Graphically:

Pick I1 (10) → 40 left
Pick I2 (20) → 20 left
Take 2/3 of I3 → done

Pseudocode

```
FractionalKnapsack(items, W):
    sort items by value/weight ratio (descending)
    profit = 0

    for item in items:
        if item.weight <= W:
            profit += item.value
            W -= item.weight
        else:
            profit += item.value * (W/item.weight)
            break

    return profit
```

Time & Space Complexity

Component	Complexity
Sorting items	$O(n \log n)$
Iterating through items	$O(n)$
Total Time	$O(n \log n)$
Space	$O(1)$

Use Cases

- ✓ Maximize profit with divisible goods
 - ✓ Cargo loading
 - ✓ Resource allocation
 - ✓ Bandwidth assignment in networks
-
-

★ GREEDY PROBLEM 3 — Job Sequencing with Deadlines

Theory

Given jobs with:

- profit
- deadline

Goal: maximize profit by completing jobs **within deadlines**, only **one job per unit time**.

Greedy picks jobs with **highest profit first**, then schedules them in the **latest possible free slot** before their deadline.

Diagram

Jobs:

Job	Profit	Deadline
J1	100	2
J2	19	1
J3	27	2
J4	25	1

Sorted by profit:

J1, J3, J4, J2

Slots:

Slot 1 | Slot 2

Final Arrangement:

Slot 1 → J3

Slot 2 → J1 → Max profit = 127

Pseudocode

```
JobSequencing(jobs):  
    sort jobs by profit descending  
    create slot array initialized empty  
  
    for job in jobs:  
        for t = job.deadline down to 1:  
            if slot[t] empty:  
                slot[t] = job  
                break
```

Time Complexity

Worst case:

$O(n \log n)$ (sorting) + $O(n^2)$ (checking slots)

Total = $O(n^2)$

Space = $O(n)$

Use Cases

- ✓ CPU scheduling
 - ✓ Deadline-driven task allocation
 - ✓ Manufacturing & batch processing
 - ✓ Online advertisement scheduling
-
-

★ GREEDY PROBLEM 4 — Minimum Number of Coins (Coin Change – Greedy)

Theory

Goal: make a value **V** using the **minimum number of coins**.

Greedy picks the **largest coin $\leq$ remaining value**.

Works only for **canonical coin systems** (like Indian currency).

Diagram Example

Coins: {1, 2, 5, 10}

Value: 27

Pick: 10 $\rightarrow$ 10 $\rightarrow$ 5 $\rightarrow$ 2 $\rightarrow$ total 4 coins

Pseudocode

```
MinCoins(coins, V):  
    sort coins descending  
    for coin in coins:  
        count += (V / coin)  
        V = V % coin
```

Time & Space Complexity

- Time: $O(n \log n + n) \approx O(n \log n)$
- Space: $O(1)$

Use Cases

- ✓ ATM withdrawal
 - ✓ Change-making in shops
 - ✓ Currency-based systems
-
-

GREEDY PROBLEM 5 — Kruskal's Minimum Spanning Tree (MST)

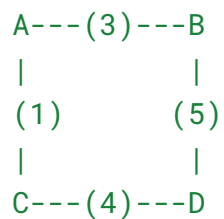
Theory

Kruskal's Algorithm builds an MST by:

1. Sorting edges by **weight**.
 2. Picking smallest edge that does **not** form a cycle.
 3. Using **Union-Find** for cycle detection.
-

Diagram

Graph:



Sorted edges:

1, 3, 4, 5

MST includes:

C-A, A-B, C-D

Pseudocode

```
Kruskal(graph):  
    result = {}  
    sort edges by weight  
    makeSet(V)  
  
    for edge in edges:  
        if find(edge.u) != find(edge.v):  
            result.add(edge)  
            union(edge.u, edge.v)
```

Time Complexity

Operation	Complexity
Sorting edges	$O(E \log E)$
Union-Find operations	$O(E \alpha(V))$ ($\approx$ constant)
Total	$O(E \log E)$
Space	$O(V)$

Use Cases

- ✓ Network design
- ✓ Minimum cost road/rail layout

- ✓ Electrical circuit design
- ✓ Cluster analysis in ML

✓ GREEDY PROBLEM 1 — Dijkstra's Algorithm (Single Source Shortest Path)

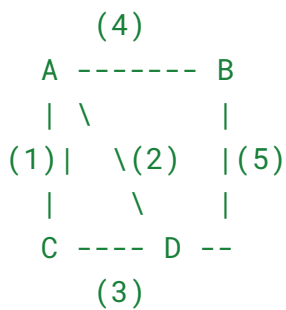
Theory

Dijkstra's Algorithm finds the **shortest path from a single source** to all other vertices in a **weighted graph with non-negative weights**.

It is a GREEDY algorithm because:

- At each step it picks the **vertex with minimum tentative distance** (greedy choice).
 - Once a node is finalized, its shortest path cannot reduce (optimal substructure).
-

Diagram



Source = A

Dry Run Table

Step	Picked Vertex	Final Distances (A→)
1	A	A=0, B=4, C=1, D=3
2	C	D=3, B stays 4
3	D	B = 4
4	B	done

Pseudocode

```

Dijkstra(Graph, source):
    dist[] = ∞
    dist[source] = 0
    priority_queue pq
    pq.push({0, source})

    while pq not empty:
        (d, u) = pq.pop_min()

        for each (u, v, w) in edges:
            if dist[v] > dist[u] + w:
                dist[v] = dist[u] + w
                pq.push({dist[v], v})

```

Time & Space Complexity

Implementation	Complexity
Using Min-Heap	$O(E \log V)$
Using Array	$O(V^2)$
Space	$O(V)$

Use Cases

- ✓ Google Maps shortest route
- ✓ Internet routing protocols
- ✓ Network packet routing
- ✓ Graph-based pathfinding in games

✓ GREEDY PROBLEM 2 — Interval Scheduling (Scheduling Problem)

Theory

Given intervals with start and finish times, choose the **maximum number of non-overlapping intervals**.

It uses the same idea as activity selection:

- Sort intervals by **finish time**
- Select earliest finishing interval

This represents **machine scheduling**, **classroom scheduling**, etc.

Diagram

Intervals:

I1: (1, 4)

I2: (3, 5)

I3: (0, 6)

I4: (5, 7)

I5: (8, 9)

Timeline:

0---6 (I3)

1--4 (I1)

3---5 (I2)

5--7 (I4)
8--9 (I5)

Selected: I1, I4, I5

Pseudocode

```
IntervalScheduling(intervals):  
    sort by finish time  
    select first interval  
    for each interval i:  
        if i.start >= last.finish:  
            select i
```

Time & Space Complexity

- Time = $O(n \log n)$
 - Space = $O(1)$
-

Use Cases

- ✓ Meeting room scheduling
- ✓ Resource allocation
- ✓ Task scheduling with deadlines

GREEDY PROBLEM 3 — Scheduling on Multiple Machines (Greedy Load Balancing)

Theory

Given **tasks** with processing times and **k machines**, schedule tasks to minimize finishing time (makespan).

Greedy method:

- Sort tasks in descending time
- Assign each task to the machine with **minimum current load**

This is called **Largest Processing Time First (LPT)**.

Diagram Example

Tasks: 8, 7, 6, 5, 4

Machines: M1, M2, M3

Step 1: assign 8 → M1 = 8

Step 2: assign 7 → M2 = 7

Step 3: assign 6 → M3 = 6

Step 4: assign 5 → M3 = 11

Step 5: assign 4 → M2 = 11

Final loads:

M1: 8

M2: 11

M3: 11

Pseudocode

```
LPT(tasks, k):  
    sort tasks descending  
    create min-heap machines of size k  
    for task in tasks:
```

```
machine = pq.pop()
machine.load += task
pq.push(machine)
```

Time Complexity

- Sorting tasks = $O(n \log n)$
 - Heap operations = $O(n \log k)$
 - Total = $O(n \log n + n \log k)$
-

Use Cases

- ✓ Cloud task scheduling
 - ✓ CPU multi-core scheduling
 - ✓ Manufacturing batch assignment
-
-

GREEDY PROBLEM 4 — Optimal Merge Pattern (File Merge Cost)

Theory

Given file sizes, merging two files costs **sum of their sizes**.

Goal: merge all files at minimum total cost.

Greedy solution:

- Always merge **two smallest** files first

- Use a **min-heap**

This is same concept as **Huffman coding**.

Diagram Example

Files: 2, 3, 4, 6

Pick smallest $(2 + 3) = 5$

Remaining: 4, 5, 6

Pick smallest $(4 + 5) = 9$

Remaining: 6, 9

Pick smallest $(6 + 9) = 15$

Total cost = $5 + 9 + 15 = 29$

Pseudocode

```
OptimalMerge(files):  
    minheap = all files  
    cost = 0  
    while heap size > 1:  
        a = heap.pop()  
        b = heap.pop()  
        merge = a + b  
        cost += merge  
        heap.push(merge)  
    return cost
```

Time Complexity

- Heap insertions = $O(n \log n)$

- Total = $O(n \log n)$
-

Use Cases

- ✓ File compression
 - ✓ External sorting
 - ✓ Multi-way merge in DBMS
-
-

GREEDY PROBLEM 5 — Huffman Coding (Optimal Prefix Codes)

Theory

Huffman Coding is used for **lossless data compression**.

Steps:

1. Compute frequencies.
2. Build a min-heap.
3. Extract two smallest frequencies.
4. Create a parent node = sum of frequencies.
5. Repeat until one root remains.

Left edge = 0, Right edge = 1.

Diagram

Characters & Frequencies:

A: 5
B: 9
C: 12
D: 13
E: 16
F: 45

Tree Constructed:



Codes generated:

A = 1100
B = 1101
C = 100
D = 101
E = 111
F = 0

Pseudocode

```
Huffman(freq):
    create minheap nodes
    while heap.size > 1:
        left = heap.pop()
        right = heap.pop()
        newnode = Node(left.freq + right.freq)
        newnode.left = left
```

```
newnode.right = right
heap.push(newnode)
return heap.pop()
```

Time Complexity

- Heap operations = $O(n \log n)$
 - Tree generation = $O(n)$
➡ Total = $O(n \log n)$
-

Use Cases

- ✓ ZIP file compression
- ✓ JPEG encoding
- ✓ Text compression
- ✓ Compiler token optimization

Greedy Coloring of Graphs — Detailed 5-Marks Answer

1. Theory

Graph coloring means assigning **colors to each vertex** of a graph such that **no two adjacent vertices share the same color**. The objective is to use the **minimum number of colors**, called the **chromatic number**, but finding this minimum is **NP-Hard**.

The **Greedy Coloring algorithm** is a heuristic that colors vertices one by one and assigns the **smallest available color** that is not used by its neighbors.

It **does not guarantee minimum colors**, but it is fast and works well in practice.

Key Idea

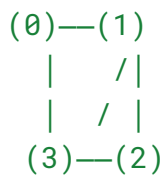
For each vertex:

1. Look at the colors used by its neighbors.
2. Assign the **lowest-numbered color** not present among neighbors.

The algorithm depends on **vertex ordering** → different orderings may use different number of colors.

2. Example With Diagram

Consider this graph:



Step-by-step Greedy Coloring

Vertex	Adjacent to	Colors used	Assigned color
0	1, 3	—	Color 0
1	0, 2	{0}	Color 1
2	1, 3	{1}	Color 0
3	0, 2	{0}	Color 1

Final coloring:

- Vertices {0, 2} → Color 0
- Vertices {1, 3} → Color 1

✓ Graph is colored with **2 colors** (not necessarily minimum always).

3. Pseudocode

```
GreedyColoring(Graph G):
    n = number of vertices
    color[n] = {-1, -1, ...}    // initially uncolored

    for each vertex u in order:
        available = array of n values initialized to true

        for each neighbor v of u:
            if color[v] != -1:
                available[color[v]] = false

        // find first available color
        for c from 0 to n-1:
            if available[c] == true:
                color[u] = c
                break

    return color[]
```

4. Time & Space Complexity

Time Complexity

- Outer loop: runs for V vertices.
- For each vertex, we scan all neighbors $\rightarrow$ total operations = $O(V + E)$

Worst-case:

$O(V + E)$

If adjacency matrix is used:

$O(V^2)$

Space Complexity

- Color array: $O(V)$
- Auxiliary availability array: $O(V)$

So total:

✓ $O(V)$

5. Use Cases / Applications

Greedy Graph Coloring is widely used where **conflict-free assignment** is required.

Applications

1. **Register allocation in compilers**
Assign CPU registers to variables without conflict.
2. **Scheduling problems**
e.g., assigning timeslots to university exams so that students with clashes don't overlap.
3. **Mobile/Radio Frequency Allocation**
Prevent interference between nearby towers.
4. **Map Coloring**
Ensures adjacent regions use different colors.
5. **Task scheduling with resource conflicts**
Useful in parallel processing.

ADVANCED ALGORITHMS

TOPIC 1 — NP-COMPLETE PROBLEMS

(Theory + examples + reduction + properties)

1.1 Define P, NP, NP-Hard, NP-Complete

Theory

- **P**: Set of decision problems solvable in polynomial time.
- **NP**: Set of problems whose solutions can be *verified* in polynomial time.
- **NP-Hard**: At least as hard as any NP problem (not required to be in NP).
- **NP-Complete**:
A problem X is NP-Complete if:

1. $X \in \text{NP}$
2. $\forall Y \in \text{NP}, Y \leq_p X$ (every NP problem reduces to X in polynomial time)

Meaning: NP-Complete problems are the hardest problems in NP.

1.2 Properties of NP-Complete Problems

- No polynomial-time algorithm is known.
- If *any* NP-Complete problem is solved in polynomial time, then:

$P = \text{NP}$

- Polynomial-time reduction is the main technique for proving NP-Completeness.
-

1.3 Classic NP-Complete problems

- **SAT** (first NP-complete by Cook-Levin Theorem)

- **Subset Sum**
 - **3-SAT**
 - **Vertex Cover (decision)**
 - **Clique (decision)**
 - **Hamiltonian Cycle**
 - **Traveling Salesman (decision version)**
-

1.4 How to Prove NP-Completeness

To prove a problem **A** is NP-Complete:

1. **Show $A \in NP$**
(i.e., verify given certificate in polynomial time)
2. **Reduce a known NP-Complete problem B to A**
Must show:

$$B \leq_p A$$

1.5 Example — Vertex Cover is NP-Complete

Reduction from 3-SAT $\rightarrow$ VC.

 **NP-Complete Problems — Full Detailed Answer**

1. Computational Complexity Classes

To understand NP-Complete problems, we need definitions of P, NP, NP-Hard, NP-Complete.

1.1 Class P

Set of decision problems that can be **solved** in polynomial time.

Example:

- Sorting
- BFS/DFS
- Shortest path in unweighted graph ($O(V + E)$)

Formally:

$P = \{ L \mid L \text{ can be decided in polynomial time } O(n^k) \}$

1.2 Class NP

Set of decision problems whose solutions can be **verified** in polynomial time.

Meaning:

Even if solving is hard, checking correctness of a solution is easy.

Formally:

$NP = \{ L \mid \text{given a certificate } C, \text{ a verifier can check } (x, C) \text{ in polynomial time} \}$

Examples:

- Checking if a graph has a Hamiltonian cycle

- Checking if a set is a vertex cover
 - Checking if there is a satisfying assignment (SAT)
Verification is polynomial, even if finding the solution is not.
-

2. NP-Hard and NP-Complete

2.1 NP-Hard

A problem is NP-Hard if it is **at least as hard as every NP problem**.

Formally:

A is NP-Hard if for every $B \in \text{NP}$, $B \leq_p A$

Meaning:

All NP problems can be reduced to A in polynomial time.

NP-Hard problems may/may not be in NP.

Examples:

- Optimization version of TSP
 - Halting Problem (not even decidable)
-

2.2 NP-Complete

A problem X is NP-Complete if:

1) $X \in \text{NP}$

2) $\forall Y \in \text{NP}, Y \leq_p X$

Meaning:

X is one of the *hardest problems in NP*.

If we find a polynomial-time algorithm for one NP-Complete problem $\rightarrow \mathbf{P = NP}$.

3. Polynomial Time Reduction (Very important)

To prove $A \leq_p B$:

- Convert input of A to input of B
- In polynomial time
- Yes/no answers must match

Diagram

Instance of A	--f(x)-->	Instance of B
YES		YES
NO		NO

This shows B is at least as hard as A.

4. Historic Result – Cook-Levin Theorem

This proved the **first NP-Complete problem**:

SAT (Boolean Satisfiability Problem) is NP-Complete.

After proving SAT is NP-Complete, many other problems were shown to be NP-Complete by reducing SAT to them.

5. Popular NP-Complete Problems (with explanation)

Below are the most important NP-Complete problems that are usually asked.

★ 5.1 SAT (Boolean Satisfiability)

Problem: Given a Boolean formula, is there an assignment of true/false values that makes it true?

Example:

$$(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3)$$

This is satisfiable.

Why NP-Complete?

- In NP: easy to verify a given assignment
- NP-Hard: Cook-Levin reduction from Turing machines

Applications

- Compiler design
 - Model checking
 - Hardware verification
 - AI constraint solving
-

★ 5.2 3-SAT (CNF with 3 literals per clause)

Example:

$(x_1 \vee x_3 \vee \neg x_4) \wedge (\neg x_1 \vee x_2 \vee x_4)$

3-SAT is NP-Complete by reduction from SAT.

3-SAT acts as a base for many reductions.

★ 5.3 CLIQUE Problem

Problem: Given a graph G and integer k , does G contain a clique of size $\geq k$?

Clique = set of vertices where each pair is connected.

Why NP-Complete?

Reduction: 3-SAT $\rightarrow$ Clique

Diagram

For each clause, create nodes for each literal.
Connect compatible literals between clauses.

★ 5.4 Vertex Cover Problem (Decision Version)

Problem: Does there exist a vertex cover of size $\leq k$?

Vertex Cover = set of vertices that touch every edge.

Why NP-Complete?

- In NP: verifying a vertex cover is easy
 - NP-Hard: reduce Clique or 3-SAT $\rightarrow$ VC
-

★ 5.5 Subset Sum

Problem: Given set S and integer T , is there a subset whose sum is T ?

Example:

$S = \{3, 8, 4, 2, 5\}$

$T = 9$

Subset = $\{4, 5\}$

Why NP-Complete?

Reduction from 3-SAT.

★ 5.6 Traveling Salesman Problem (TSP) – Decision Version

Problem:

Given distances and value B , is there a tour of cost $\leq B$?

Why NP-Complete?

Reduction: Hamiltonian Cycle $\rightarrow$ TSP

Important:

Optimization version = NP-Hard

Decision version = NP-Complete

★ 5.7 Hamiltonian Cycle Problem

Problem: Does a graph contain a cycle visiting each vertex exactly once?

Why NP-Complete?

Reduction from 3-SAT.

★ 5.8 Partition Problem

Problem: Can we split set S into two subsets with equal sum?

Example:

$\{1, 5, 11, 5\} \rightarrow \text{YES } (11 = 5+5+1)$

★ 5.9 Knapsack Problem (0/1 Knapsack Decision)

Problem:

Is there a subset of items with value $\geq V$ and weight $\leq W$?

6. Proving NP-Completeness — Complete Steps (Exam Answer)

To prove a new problem X is NP-Complete:

Step 1: Show $X \in \text{NP}$

Given a solution (certificate), verify it in polynomial time.

Example:

Given a set of vertices claimed to be a vertex cover $\rightarrow$ check edges.

Step 2: Choose a known NP-Complete problem Y

Common choices:

- 3-SAT
 - Clique
 - Vertex Cover
 - Subset Sum
-

Step 3: Reduce Y to X

Construct a polynomial-time mapping:

$f : \text{Instances of Y} \rightarrow \text{Instances of X}$

Step 4: Prove correctness

Show:

$y \text{ is YES-instance} \Leftrightarrow f(y) \text{ is YES-instance}$

Step 5: Conclude

Since:

- $X \in \text{NP}$

- $Y \leq_p X$

Therefore:

X is NP-Complete

7. Why NP-Complete Problems Matter (Applications)

Even though NP-Complete problems are hard, they appear everywhere:

♦ Computer Networks

- Routing
- Scheduling packets

♦ Database Systems

- Query optimization
- Transaction scheduling

♦ AI & Machine Learning

- Feature selection
- Neural architecture search
- Constraint satisfaction

♦ Cyber Security

- Cryptographic hardness assumptions

◆ Operations Research

- Logistics optimization
- Vehicle routing
- Warehouse scheduling

◆ Compiler Design

- Register allocation
- Code optimization

8. Why NP-Complete Problems Are Difficult

No polynomial-time algorithm is known.

If any NP-Complete problem is solved in polynomial time:

$P = NP$

This would break modern cryptography (RSA, AES, ECC, etc.).

9. Typical Exam Diagrams

Vertex Cover Example

```
a----b
|    /|
|   / |
c----d
```

Minimum VC = {b, c}

Hamiltonian Cycle Example

A - B - C
| |
F - E - D

Cycle: A-B-C-D-E-F-A

SAT → 3-SAT → Clique Diagram

Each clause: $(x \vee y \vee \neg z)$

Create nodes for each literal.

Connect compatible literals from different clauses.

10. Time Complexity Notes

All NP-Complete decision problems:

- No known polynomial solution
- Require exponential worst-case time for exact solution

Approximation or heuristics are often used instead.

★ TOPIC 2 — APPROXIMATION ALGORITHMS (5–10 marks)

2.1 What is an Approximation Algorithm?

Since optimal solution is expensive for NP-Hard problems, approximation gives near-optimal solution.

Approximation Ratio

For minimization:

$$\rho = A / OPT$$

For maximization:

$$\rho = OPT / A$$

2.2 Approximation Algorithm for Vertex Cover (2-Approx)

Algorithm

1. Pick an arbitrary uncovered edge (u, v)
2. Add both u and v to Vertex Cover
3. Remove all edges touching them
4. Repeat

Approximation Guarantee

Returned cover A satisfies:

$$|A| \leq 2 * |OPT|$$

Time Complexity: $O(E)$

2.3 Approximation Algorithm for TSP (Metric TSP – 2 Approx)

Steps:

1. Compute MST
2. Perform preorder walk (DFS traversal)
3. Shortcut repeated vertices

Cost Bound

$$\text{Tour_cost} \leq 2 * \text{MST_cost} \leq 2 * \text{OPT}$$

Time Complexity: $O(E \log V)$

TOPIC 3 — ALGORITHMS FOR MATCHING

(Already given but here is an organized version for your final notes.)

3.1 Bipartite Maximum Matching (Augmenting Path)

Key Concept

Augmenting path alternates between unmatched and matched edges.

Diagram

```
L = {a,b,c}
R = {1,2,3}
```

```
a -- 1
a -- 2
b -- 1
c -- 2
c -- 3
```

3.2 Pseudocode

(DFS-based Kuhn algorithm)

```
dfs(u):
    for each v in adj[u]:
        if not visited[v]:
            visited[v] = true
            if matchR[v] == 0 or dfs(matchR[v]):
                matchR[v] = u
                return true
    return false
```

Time Complexity

$O(V * E)$

3.3 Applications

- Job assignment
- Course allocation
- Stable scheduling
- Maximum partners in bipartite networks

- Network flow pre-processing

★ TOPIC 4 — FLOW PROBLEMS

4.1 MAXIMUM FLOW — Ford–Fulkerson

Goal

Find max possible flow from **source (s)** to **sink (t)** in a directed graph.

Residual Graph

For each edge $(u \rightarrow v)$:

```
capacity    =  $c(u, v)$ 
flow        =  $f(u, v)$ 
residual    =  $c(u, v) - f(u, v)$ 
reverse     =  $f(u, v)$       (to undo flow)
```

4.2 Algorithm Steps

1. Initialize flow = 0
2. Build residual graph
3. Find augmenting path using DFS/BFS
4. Increase flow
5. Update residual graph

6. Repeat until no path exists

4.3 Example graph

$s \rightarrow a \rightarrow t$
 $s \rightarrow b \rightarrow t$
 $a \rightarrow b$

4.4 Max Flow-Min Cut Theorem

Maximum Flow = Minimum Capacity of s-t Cut

4.5 Complexity

- With BFS (Edmonds–Karp):

$O(V * E^2)$

★ TOPIC 5 — CIRCULATION WITH DEMANDS (Flow + lower bounds)

5.1 Circulation Definition

Circulation assigns flows to edges such that:

Incoming flow = Outgoing flow

for every vertex, and flows must respect:

$$\text{lower_bound} \leq \text{flow} \leq \text{capacity}$$

5.2 Steps to Solve Circulation with Demands

1. Convert each edge $(u \rightarrow v)$ with lower bound $l(u, v)$ and capacity $c(u, v)$ to a modified capacity:

$$\text{capacity}' = c(u, v) - l(u, v)$$

2. Maintain demand for each vertex:

$$\text{demand}(v) = \text{sum}(l(\text{incoming})) - \text{sum}(l(\text{outgoing}))$$

3. Add **super-source S** and **super-sink T**
 4. Connect $S \rightarrow v$ for positive demands
 5. Connect $v \rightarrow T$ for negative demands
 6. Check if max-flow saturates all edges from S
-

Use Cases

- Circulations in logistics
 - Supply–demand balancing
 - Lower-bound flow constraints
-

★ TOPIC 6 — BIO-INSPIRED ALGORITHMS

(Very important for theory)

6.1 Genetic Algorithm (GA)

Theory

GA mimics natural selection. Solutions = chromosomes.

Steps

1. Initialize population
2. Evaluate fitness
3. Selection
4. Crossover
5. Mutation
6. Replacement

Fitness Function

`fitness(x) = quality(x)`

Applications

- TSP
- Knapsack
- Neural architecture search

- Feature selection
-

6.2 Particle Swarm Optimization (PSO)

Inspired by: Bird flocking / fish schooling

Equation

Position update:

$$v = w*v + c1*r1*(pbest - x) + c2*r2*(gbest - x)$$
$$x = x + v$$

Use

- Continuous optimization
 - ML model tuning
-

6.3 Artificial Bee Colony (ABC)

Inspired by: Foraging behavior of honey bees.

Three Bee Types

- Employed
 - Onlooker
 - Scout
-

6.4 Firefly Algorithm

Attractiveness Equation

$$\beta = \beta_0 * \exp(-\gamma * r^2)$$

Uses brightness, attractiveness, distance.

6.5 Bat Algorithm

Equations

Velocity & position update:

$$\begin{aligned} v &= v + (x - gbest) * f \\ x &= x + v \end{aligned}$$

Loudness update:

$$A(t+1) = \alpha * A(t)$$

★ N-Queens Problem

1. Introduction & Theory

The **N-Queens problem** is a classical **backtracking** and **constraint-satisfaction** problem in which we must place **N queens on an N×N chessboard** such that:

1. No two queens are in the **same row**
2. No two queens are in the **same column**
3. No two queens are on the **same diagonal**

This means each queen must attack **zero** other queens.

✓ Problem definition

Place N queens on an N×N board such that:

No two queens share:

- Row
- Column
- Major diagonal ($r - c$ same)
- Minor diagonal ($r + c$ same)

The problem is important because it demonstrates:

- Constraint propagation
- Backtracking search
- Pruning invalid states
- NP-hard search space for large N

★ 2. Why Backtracking?

Brute force tries all possible placements:

Number of ways to place N queens on N×N grid:

$(N \times N \text{ choose } N)$

This is exponential and infeasible.

Backtracking prunes:

- If a queen placement is invalid → STOP exploring that branch
- Only place queen in safe columns
- Move row by row

Thus, backtracking reduces the search drastically.

3. Board Representation

Common representation:

Use a 1D array `board[ ]` where:

`board[row] = column where queen is placed`

Example for N=4:

```
Row:    0 1 2 3
Col:    1 3 0 2
```

Means:

- Q1 at (0,1)
- Q2 at (1,3)
- Q3 at (2,0)

- Q4 at (3,2)
-

★ 4. Safe Position Check

A queen at (row, col) is unsafe if:

Same column: `board[i] == col`
Major diagonal: `(i - board[i]) == (row - col)`
Minor diagonal: `(i + board[i]) == (row + col)`

Diagram — Unsafe Attacking Positions

If queen is at:

(2, 3)

Then unsafe squares:

Row 2: Q Q Q Q ...
Column 3: | | Q | |
Diag ↘: positions where r-c constant = -1
Diag ↗: positions where r+c constant = 5

★ 5. Backtracking Algorithm Steps (Concept)

1. Start at row = 0
2. Try placing queen in col = 0 to N-1
3. If the position is safe → place queen

4. Move to next row
 5. If all N queens placed → solution
 6. If no column works → backtrack, remove previous queen
-

★ 6. Pseudocode (Exam Format)

```
FUNCTION solveNQueens(board, row, N):
    IF row == N:
        PRINT board    // A solution found
        RETURN

    FOR col = 0 to N-1:
        IF isSafe(board, row, col):
            board[row] = col
            solveNQueens(board, row + 1, N)
            board[row] = -1    // backtrack

FUNCTION isSafe(board, row, col):
    FOR i = 0 to row-1:
        IF board[i] == col:
            RETURN false
        IF (i - board[i]) == (row - col):
            RETURN false
        IF (i + board[i]) == (row + col):
            RETURN false
    RETURN true
```

★ 7. C++ Code (Optimized Backtracking)

```
#include <iostream>
#include <vector>
using namespace std;
```

```

bool isSafe(vector<int>& board, int row, int col) {
    for (int i = 0; i < row; i++) {
        if (board[i] == col) return false;           //
same column
        if (abs(board[i] - col) == abs(i - row)) return false; //
diagonal
    }
    return true;
}

void solveNQueens(vector<int>& board, int row, int n) {
    if (row == n) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++)
                cout << (board[i] == j ? "Q " : ". ");
            cout << "\n";
        }
        cout << "\n";
        return;
    }

    for (int col = 0; col < n; col++) {
        if (isSafe(board, row, col)) {
            board[row] = col;
            solveNQueens(board, row + 1, n);
            board[row] = -1;    // backtrack
        }
    }
}

int main() {
    int n = 8;
    vector<int> board(n, -1);
    solveNQueens(board, 0, n);
    return 0;
}

```

★ 8. Worked Example (N = 4)

Step-by-step:

1. Place Q1 in row 0
Try col 0 → safe
2. Row 1
col 0 → same column ✗
col 1 → diagonal conflict ✗
col 2 → safe
3. Row 2
col 0 → diagonal conflict ✗
col 1 → safe
4. Row 3
col 0 → conflict
col 1 → conflict
col 2 → conflict
col 3 → safe

One solution:

```
. Q . .  
. . Q .  
Q . . .  
. . . Q
```

★ 9. Time Complexity

For each row, N choices → worst case:

Time Complexity = $O(N!)$

Because the first row has N choices, second row N-1, and so on.

More formally:

$$T(N) \approx N \times (N-1) \times (N-2) \times \dots \times 1 = O(N!)$$

Space Complexity

$$O(N)$$

For the board array + recursion depth.

10. Real-World Applications

The N-Queens problem models many **constraint satisfaction** and **optimization** tasks:

1. Scheduling Systems

- Place entities without conflicts
- Example: exam timetabling
- Avoids collisions (similar to queens not attacking)

2. VLSI Design (Chip Layout)

- Minimize interference
- Arrange components such that no lines cross

3. Parallel Computing

- Task allocation without resource conflicts

4. Robotics Path Planning

- Avoiding collision zones similar to diagonal conflicts

5. Artificial Intelligence

- Demonstrates constraint programming
 - Used in AI textbooks to teach backtracking and heuristics
-

★ 11. Why N-Queens Is Important in Algorithms

- Demonstrates **recursion and backtracking**
 - Shows **pruning invalid states**
 - Forms base for **Constraint Satisfaction Problems (CSPs)**
 - Introduces logical constraints similar to SAT solving
 - Used in exam questions for demonstrating:
 - DFS search
 - Backtracking tree
 - Time-space analysis
-

★ 12. Diagram of Backtracking Tree (N=4)

```
Row 0:
|-- col0
|   |-- col1 ✗
|   |-- col2
|       |-- col1
|       |   |-- col3 ✓ solution
|       |   |-- col? (backtrack)
|       |-- col3 ✗
```

```
|  
|-- col1  
|-- col2  
|-- col3
```

This shows how pruning reduces large search branches.